

Press centrum pro fotografie
Debian + Apache + PHP + MariaDB
FTP upload z fotoaparátů
watcher generuje preview
redaktoři lockují a stahují fotky
workflow: ready → locked → downloaded

Tohle dává smysl jako jednoduché interní **press centrum** s jasným workflow a minimem závislostí. Níže je návrh architektury, datového modelu a logiky provozu.

Základní architektura

Stack:

- Debian
- Apache + PHP
- MariaDB
- FTP server pro upload z fotoaparátů / editorů
- **watcher** běžící jako systemd služba nebo cron
- souborové úložiště na disku

Navržený adresářový layout

```
/var/presscentrum/  
├─ incoming/      # sem padají nové uploady přes FTP  
├─ original/      # finální originály po zařazení  
├─ preview/       # generované náhledy  
├─ locked/        # volitelně pracovní adresář  
├─ downloaded/    # volitelně archiv přesunutých souborů  
└─ logs/
```



Jednodušší varianta je soubory **nepřesouvat mezi adresáři**, ale měnit jen stav v databázi:

- originál je pořád v `/original/`
- preview v `/preview/`
- stav se řídí sloupcem v DB

To bývá praktičtější.

Workflow



Význam stavů

- **ready**
Fotka je nahraná, preview existuje, redaktor ji může vidět a zamknout.
 - **locked**
Fotka je rezervovaná konkrétním redaktorem, ostatní ji už nemají mít možnost stáhnout.
 - **downloaded**
Fotka byla skutečně stažena a workflow je uzavřené.
-

Doporučený tok zpracování

1. Upload přes FTP

Fotoaparát nebo fotograf nahraje soubor do:

```
/var/presscentrum/incoming/
```



FTP účet ideálně omezit jen na upload a bez shellu.

2. Watcher

Watcher pravidelně:

- najde nové soubory v `incoming/`
- ověří, že upload už skončil
- přesune je do `original/`
- vytvoří preview do `preview/`
- zapíše záznam do DB se stavem `ready`

3. Web pro redaktory

PHP aplikace zobrazí seznam fotek ve stavu `ready` :

- náhled
- název souboru
- čas nahrání
- autor / zdroj
- tlačítko **Lock**

4. Lock

Po kliknutí na **Lock**:

- aplikace v DB atomicky změní stav `ready` -> `locked`

- uloží locked_by , locked_at
- pokud už mezitím někdo jiný zamkl, uživatel dostane informaci, že fotka není dostupná

5. Download

Redaktor stáhne originál. Po úspěšném stažení:

- stav locked -> downloaded
- uloží se downloaded_by , downloaded_at

Databázový model

Tabulka users

```
CREATE TABLE users (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  username VARCHAR(64) NOT NULL UNIQUE,  
  password_hash VARCHAR(255) NOT NULL,  
  role ENUM('editor','admin') NOT NULL DEFAULT 'editor',
```

SQL



Tabulka photos

```
CREATE TABLE photos (  
  id BIGINT AUTO_INCREMENT PRIMARY KEY,  
  filename_original VARCHAR(255) NOT NULL,  
  filepath_original VARCHAR(500) NOT NULL,  
  filename_preview VARCHAR(255),  
  filepath_preview VARCHAR(500),  
  filesize BIGINT,  
  mime_type VARCHAR(100),  
  checksum_sha256 CHAR(64),  
  author VARCHAR(255),  
  uploaded_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  status ENUM('ready','locked','downloaded') NOT NULL DEFAULT 'ready',  
  
  locked_by INT NULL,  
  locked_at DATETIME NULL,
```

SQL



Volitelná auditní tabulka photo_log

```
CREATE TABLE photo_log (  
  id BIGINT AUTO_INCREMENT PRIMARY KEY,  
  photo_id BIGINT NOT NULL,  
  user_id INT NULL,  
  action VARCHAR(50) NOT NULL,  
  note TEXT NULL,  
  created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  
  FOREIGN KEY (photo_id) REFERENCES photos(id) ON DELETE CASCADE,
```

SQL



Akce:

- uploaded
- preview_generated
- locked
- downloaded
- unlock_forced

Důležitá věc: atomický lock

Aby dva redaktoři nezamkli stejnou fotku současně, musí být lock udělán jedním SQL dotazem:

```
UPDATE photos  
SET status = 'locked',  
    locked_by = ?,  
    locked_at = NOW()
```

SQL



Pak v PHP zkontrolovat počet ovlivněných řádků:

- 1 = lock úspěšný
- 0 = už zamknul někdo jiný

To je klíčové.

Watcher: co má dělat

Watcher může být v PHP CLI, Pythonu nebo shellu. Prakticky bych ho dal třeba jako:

- `systemd` service + timer
nebo
- cron každých 10–30 sekund

Logika watcheru

1. projít `incoming/`
2. ignorovat soubory, které se ještě zapisují
3. spočítat hash
4. přesunout do `original/`
5. vygenerovat preview
6. vložit do DB

Ověření dokončeného uploadu

Jednoduchý přístup:

- soubor musí mít stejnou velikost při dvou kontrolách po několika sekundách

Lepší přístup:

- fotograf nahrává nejdřív jako `*.part`
- po dokončení přejmenuje na finální jméno

U FTP klientů to ale nemusí být vždy snadno ovlivnitelné, takže kontrola velikosti je praktičtější.

Generování preview

Na Debianu pohodlně přes ImageMagick nebo libvips.

Příklad:

```
</> Bash
```



Doporučuji:

- zachovat EXIF orientaci
- preview s rozumnou velikostí
- originál nechat beze změn

Webová část v PHP

Stránky

- login

- seznam ready fotek
- detail fotky
- moje locked fotky
- archiv downloaded
- admin uživatelé / logy

Funkce

- filtrování podle času, autora, názvu
 - stránkování
 - náhledy v gridu
 - lock jedním klikem
 - download jen pro zamčenou fotku
 - admin možnost odemknutí
-

Pravidla přístupu

Redaktor

- vidí ready
- může zamknout
- může stáhnout jen to, co zamkl on
- vidí své locked a downloaded

Admin

- vidí vše
 - může odemknout cizí lock
 - může měnit stav
 - může mazat chybné importy
-

Bezpečnost

- FTP uživatelé bez shellu
- upload mimo webroot
- originály nestahovat přímo přes URL na disk
- download jen přes PHP kontrolující oprávnění
- hesla hashovat přes `password_hash()`
- session ochrana
- CSRF tokeny na akce lock/unlock
- logování všech stavových změn

Důležité:

preview může být veřejněji dostupné interně, ale originál jen přes autorizovaný skript.

Doporučený způsob ukládání souborů

Místo jednoho obřího adresáře:

/var/presscentrum/original/2026/03/31/



a podobně pro preview:

/var/presscentrum/preview/2026/03/31/



Je to přehlednější a lépe škáluje.

Možné rozšíření

- automatické čtení EXIF
 - IPTC metadata
 - fulltext nad popisem
 - štítky událostí
 - priorita fotek
 - dávkové lockování
 - API pro externí redakční systém
 - webhook po uploadu
-

Minimální MVP

Na první verzi bych udělal jen:

1. FTP upload do `incoming`
2. watcher:
 - přesun do `original`
 - preview do `preview`
 - DB záznam `ready`
3. PHP login
4. galerie `ready`
5. atomický lock
6. download jen pro zamčenou fotku
7. přechod do `downloaded`

To už bude plně použitelné.

Doporučení k implementaci

Nejčistší je:

- stavy držet v databázi
- soubory nepřesouvat podle workflow
- fyzicky mít jen:
 - incoming
 - original
 - preview

Je to jednodušší, rychlejší a méně náchylné na chyby.

Mohu ti z toho hned připravit i konkrétní:

- SQL schéma,
- návrh adresářů a práv,
- PHP tabulky a endpointy,
- nebo rovnou první kostru aplikace.

Pokračujme Sprint 3A – bulk download

Jasně. **Sprint 3A – bulk download** bych postavil tak, aby navázal na existující workflow a nerozbil atomické lockování.

Cíl sprintu

Umožnit redaktorovi:

- vybrat více fotek najednou,
- hromadně je zamknout,
- stáhnout je v jednom ZIPu,
- po úspěšném vygenerování ZIPu je převést do stavu `downloaded`.

Workflow pořád zůstane:

ready → locked → downloaded



Jen se přidá hromadná operace nad více soubory.

1. Funkční zadání

Uživatelský scénář

Redaktor:

1. otevře galerii fotek ve stavu `ready`
2. zaškrtně více položek
3. klikne na **Bulk lock + download**
4. systém:
 - zkusí každou fotku zamknout pro aktuálního uživatele
 - fotky, které už mezitím zamkl někdo jiný, přeskočí
 - z úspěšně zamčených vytvoří ZIP
 - nabídne ZIP ke stažení
 - po dokončení přípravy ZIPu označí tyto fotky jako `downloaded`

Volitelně lze udělat i dvě tlačítka:

- **Bulk lock**
- **Download locked as ZIP**

Ale pro provoz bývá praktičtější jedno tlačítko.

2. Rozhodnutí k workflow

Jsou dvě rozumné varianty:

Varianta A – po vytvoření ZIP rovnou `downloaded`

Jakmile server úspěšně připraví ZIP, označí fotky jako `downloaded`.

Výhody

- jednoduché
- snadná implementace
- odpovídá internímu workflow „už si je redaktor vzal“

Nevýhody

- nepoznáš, jestli uživatel ZIP opravdu stáhl až do konce

Varianta B – nejdřív `locked`, po skutečném download requestu `downloaded`

- server nejprve vytvoří ZIP
- uloží job nebo dočasný soubor
- až když uživatel klikne na finální download URL, přepne stav na `downloaded`

Výhody

- přesnější

- lépe odpovídá realitě

Nevýhody

- složitější

Doporučení pro Sprint 3A

Na první iteraci bych zvolil **Variantu A**.

Je dostatečně praktická a výrazně jednodušší.

3. Co bude potřeba doplnit

Databáze

Současná tabulka `photos` může zůstat. Přidal bych ale tabulku pro dávkové akce.

Tabulka `bulk_downloads`

```
CREATE TABLE bulk_downloads (  
  id BIGINT AUTO_INCREMENT PRIMARY KEY,  
  user_id INT NOT NULL,  
  zip_name VARCHAR(255) NOT NULL,  
  zip_path VARCHAR(500) NOT NULL,  
  photo_count INT NOT NULL DEFAULT 0,  
  created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  expires_at DATETIME NULL,
```

SQL



Vazební tabulka `bulk_download_items`

```
CREATE TABLE bulk_download_items (  
  id BIGINT AUTO_INCREMENT PRIMARY KEY,  
  bulk_download_id BIGINT NOT NULL,  
  photo_id BIGINT NOT NULL,  
  
  FOREIGN KEY (bulk_download_id) REFERENCES bulk_downloads(id) ON DELETE CASCADE  
  FOREIGN KEY (photo_id) REFERENCES photos(id) ON DELETE CASCADE
```

SQL



To je užitečné pro:

- audit,

- zpětné dohledání, co bylo v ZIPu,
- případné pozdější znovustažení.

4. Adresáře

Přidal bych:

`/var/presscentrum/tmpzip/`



Práva:

- zapisovat jen uživatel, pod kterým běží Apache/PHP
- nepublikovat přímo přes webserver bez kontroly

5. Backend logika

Endpoint

Například:

`POST /bulk-download.php`



Vstup:

- pole ID fotek: `photo_ids[]`

Postup

Krok 1 – validace vstupu

- uživatel přihlášen
- seznam ID není prázdný
- maximální počet třeba 200 nebo 500 kusů

Krok 2 – načtení kandidátů

Najít všechny požadované fotky.

Krok 3 – atomické lockování po jedné fotce

Pro každé ID:

```
UPDATE photos
SET status = 'locked',
    locked_by = :user_id,
```

```
locked_at = NOW()
```

SQL



Pokud `rowCount() === 1`, lock proběhl.

Zároveň je dobré povolit i situaci:

- fotka už je `locked`
- a `locked_by` = aktuální uživatel

Takovou fotku lze do ZIPu také zahrnout.

Krok 4 – sestavení seznamu skutečně dostupných souborů

Do ZIPu půjdou:

- nově zamčené fotky
- už dříve zamčené tímto uživatelem

Nepůjdou:

- `locked` někým jiným
- `downloaded`
- neexistující soubory

Krok 5 – vytvoření ZIPu

Použít PHP `ZipArchive`.

Krok 6 – zapsání bulk download záznamu

- insert do `bulk_downloads`
- insert položek do `bulk_download_items`

Krok 7 – přepnutí do `downloaded`

Pro všechny úspěšně zahrnuté fotky:

```
UPDATE photos
SET status = 'downloaded',
    downloaded_by = :user_id,
    downloaded_at = NOW()
WHERE id = :photo_id
```

SQL



Krok 8 – stream ZIPu uživateli

- `Content-Type: application/zip`

- Content-Disposition: attachment; filename="presscentrum-2026-03-31-001.zip"
-

6. Důležitá pravidla

A. Nikdy nespoléhat jen na checkboxy z formuláře

Vždy znovu kontrolovat v DB:

- stav
- vlastníka locku
- existenci souboru

B. ZIP nesmí obsahovat kolizní názvy

Když dva soubory mají stejné jméno, je potřeba je přejmenovat uvnitř ZIPu.

Např.:

IMG_0001.jpg
IMG_0001_2.jpg
IMG_0001_3.jpg



C. Limit velikosti dávky

Aby server neumřel:

- počet souborů limitovat
- celkovou velikost třeba také kontrolovat

Např. MVP:

- max 200 souborů
- nebo max 4 GB

D. Mazání starých ZIPů

Cron:

- smaže ZIPy starší než třeba 24 hodin
-

7. Návrh UI

V galerii:

- checkbox u každé fotky
- checkbox v hlavičce „vybrat vše na stránce“
- tlačítko **Stáhnout vybrané jako ZIP**

Nad výsledky lze zobrazit shrnutí:

- vybráno: 36
- uzamčeno a staženo: 31
- přeskočeno: 5

Přeskočené kusy:

- už zamkl jiný redaktor
- soubor nenalezen

8. Návrh odpovědi systému

Po akci by bylo dobré ukázat:

ZIP byl vytvořen.
Celkem vybráno: 24
Zařazeno do ZIPu: 21
Přeskočeno: 3



A pod tím třeba seznam přeskočených položek.

9. Doporučená implementace v PHP

Příklad kostry `bulk-download.php`

```
<?php
declare(strict_types=1);

session_start();

require __DIR__ . '/config.php';
require __DIR__ . '/db.php';
require __DIR__ . '/auth.php';

$userId = require_login();

if ($_SERVER['REQUEST_METHOD'] !== 'POST') {
    http_response_code(405);
    exit('Method Not Allowed');
}
```

PHP



10. Poznámka k transakci

Výše uvedená verze drží DB transakci docela dlouho, což u větších ZIPů není ideální.

Lepší produkční varianta:

1. v transakci zamknout a vybrat kandidáty
2. commit
3. vytvořit ZIP mimo transakci
4. v nové krátké transakci označit `downloaded`

To je robustnější.

Pro MVP to ale může fungovat, pokud bude dávka menší.

11. Lepší produkční návrh

Fáze 1 – rezervace

- v DB zamknout položky pro uživatele

Fáze 2 – build ZIP

- mimo transakci vytvořit ZIP

Fáze 3 – potvrzení downloadu

- zapsat `bulk_download`
- přepnout vybrané fotky na `downloaded`

To bych doporučil jako cílový stav.

12. Co ošetřit ve Sprintu 3A

Hotovo v tomto sprintu

- výběr více fotek
- hromadné lockování
- ZIP download
- přechod do `downloaded`
- audit bulk akce
- cleanup starých ZIPů

Nechat až do dalšího sprintu

- background build ZIPů
- progress bar

- opakované stažení starého ZIPu
 - dělení velkých ZIPů na více archivů
 - partial retry
-

13. Akceptační kritéria

Sprint 3A je hotový, když:

1. redaktor může vybrat více `ready` fotek
 2. systém zamkne jen ty, které jsou skutečně volné
 3. položky zamčené jiným uživatelem se nepřidají
 4. systém vytvoří validní ZIP
 5. po akci jsou zahrnuté fotky ve stavu `downloaded`
 6. akce se zapíše do audit logu
 7. duplicitní názvy uvnitř ZIPu nezpůsobí chybu
-

14. Doporučení k dalšímu kroku

Za mě je ideální další konkrétní krok:

- udělat **DB migraci**
- připravit **PHP endpoint**
- doplnit **checkboxy do galerie**
- přidat **cron na mazání starých ZIPů**

Mohu ti hned napsat i **Sprint 3A jako přesný implementační checklist**, případně rovnou dodat **SQL migraci + finální PHP soubor + HTML formulář pro galerii**.

Se vším souhlasím, jen by bylo lepší bulk download neřešit jako zip (obrázky RAW dnes už většinou mají bezetrátovou kompresi a zip už je tak o moc nezmenší a navíc to přidá zbytečný mezikrok pro redaktora, který je bude muset po downloadu zase rozzipovávat). Lepší by bylo, kdyby se vytvořila jakási fronta downloadů a browser si je stáhnul pěkně v řadě za sebou. Lze něco takového udělat?

Ano, **udělat to lze** a pro váš use-case je to často lepší než ZIP. Jen je potřeba počítat s jedním praktickým omezením: **prohlížeče nerady dovolují „automaticky“ spustit desítky**

samostatných downloadů bez interakce uživatele. Takže řešení existuje, ale musí být navržené chytrě.

Stručně

Místo ZIPu bych pro **Sprint 3A** navrhl:

- uživatel vybere více fotek,
- server provede **bulk lock**,
- vytvoří se **download job / fronta**,
- klientská stránka pak postupně stahuje jednotlivé soubory:
 - buď přes sérii odkazů / iframe,
 - nebo přes postupné otevírání download URL,
- po každém potvrzeném vydání souboru se fotka přepne do `downloaded` .

To je reálné a architektonicky čisté.

Co je na tom dobré

Výhody oproti ZIPu

- žádné rozbalování
- redaktor dostane rovnou originály
- žádné dočasné velké ZIP soubory na serveru
- žádný čas na kompresi
- menší zátěž CPU

Nevýhody

- browser může část downloadů blokovat
 - UX bude trochu citlivější na nastavení prohlížeče
 - je potřeba řešit „co už bylo staženo“ po jednotlivých souborech
-

Ano, jde to udělat – ale ve dvou vrstvách

1. Serverová vrstva

Server vytvoří **download frontu**:

- které fotky byly vybrány,
- které se podařilo locknout,
- které už byly vydány,
- které přeskočit.

2. Klientská vrstva

Browser pak z této fronty bere soubory **jeden po druhém**.

Doporučený návrh

Nový workflow pro bulk

Namísto:

ready → locked → downloaded



zůstane stejné, ale po položkách:

1. vybrané fotky se hromadně zamknou
2. vznikne `download_job`
3. klient spouští download souborů jeden po druhém
4. po odeslání konkrétního souboru server přepne jen ten jeden na `downloaded`

To je přesnější než ZIP.

Datový model

Tabulka `download_jobs`

```
CREATE TABLE download_jobs (  
  id BIGINT AUTO_INCREMENT PRIMARY KEY,  
  user_id INT NOT NULL,  
  status ENUM('prepared', 'running', 'finished', 'failed') NOT NULL DEFAULT 'prepared',  
  created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  started_at DATETIME NULL,  
  finished_at DATETIME NULL,  
  
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
```

SQL



Tabulka `download_job_items`

```
CREATE TABLE download_job_items (  
  id BIGINT AUTO_INCREMENT PRIMARY KEY,  
  job_id BIGINT NOT NULL,  
  photo_id BIGINT NOT NULL,  
  seq_no INT NOT NULL,
```

```
status ENUM('queued','downloading','downloaded','skipped','failed') NOT NULL
reason VARCHAR(100) NULL,
locked_at DATETIME NULL,
downloaded_at DATETIME NULL,

FOREIGN KEY (job_id) REFERENCES download_jobs(id) ON DELETE CASCADE,
FOREIGN KEY (photo_id) REFERENCES photos(id) ON DELETE CASCADE,
UNIQUE KEY uniq_job_seq (job_id, seq_no),
```

SQL



To je podle mě lepší než původní `bulk_downloads`, protože to přesně odpovídá frontě.

Jak by to fungovalo

Krok 1 – uživatel vybere fotky

Např. v galerii zaškrtně 40 položek.

Krok 2 – server vytvoří job

Endpoint:

POST `/bulk-download-create.php`



Server:

- projde zadaná ID,
- pokusí se každou fotku locknout,
- co jde zamknout, vloží do `download_job_items` jako `queued`,
- co nejde, vloží jako `skipped`,
- vrátí `job_id`.

Krok 3 – browser otevře stránku „Probíhá stahování“

Např.:

`/download-queue.php?job=123`



Tato stránka obsahuje JavaScript, který začne postupně spouštět jednotlivé downloady.

Krok 4 – jednotlivé soubory se vydávají po jednom

Endpoint:

GET `/download-item.php?job=123&item=456`



Ten:

- ověří přihlášení,
 - ověří, že job patří uživateli,
 - ověří, že položka je `queued`,
 - že fotka je `locked` tímto uživatelem,
 - pošle soubor,
 - označí položku jako `downloaded`,
 - a zároveň `photos.status = downloaded`.
-

Klíčová otázka: půjde browser opravdu stahovat „sám v řadě“?

Ano, ale s omezením

Technicky lze:

- vytvořit sérii skrytých `iframe`,
- nebo postupně kliknout na dočasně vytvořené `<a download>`
- nebo otevírat jednu URL po druhé.

Ale:

Prohlížeče mohou vícenásobné downloads blokovat

Typicky:

- Chrome / Edge / Firefox často dovolí první download
- u dalších mohou chtít potvrzení
- nebo uživatel musí povolit „multiple automatic downloads“

Takže:

- **technicky ano**
- **stoprocentně bez zásahu uživatele ne vždy**

To je nejdůležitější praktická poznámka.

Co s tím

Doporučené UX

Po kliknutí na „Stáhnout vybrané“ zobrazit stránku:

Připraveno ke stažení: 27 souborů.

Váš prohlížeč může požádat o povolení více souborů.



Pokud se stahování zastaví, klikněte na „Pokračovat“.

A pod tím:

- progress bar
- počítadlo
- tlačítko **Pokračovat**
- tlačítko **Stáhnout další soubor**

Takže ideální je **automat + možnost ručního pokračování**.

Doporučená implementace Sprint 3A

Varianta A – automatická fronta s fallbackem

To bych doporučil nejvíc.

Chování

- JS se pokusí spouštět downloady po jednom
- pokud browser pustí vše, super
- pokud se zasekne, uživatel klikne na **Pokračovat**
- fronta pokračuje od další položky

To je velmi praktické.

Serverové endpointy

1. Vytvoření fronty

POST /bulk-download-create.php



Vrátí třeba JSON:

```
{
  "job_id": 123,
  "queued": 27,
  "skipped": 3,
```

JSON



2. Stav fronty

GET /api/download-job-status.php?job=123



Vrátí:

- celkem
- queued
- downloaded
- skipped
- next item

3. Vydání jedné položky

GET /download-item.php?job=123&item=456



Vrátí soubor jako attachment.

4. Označení chyby / skip

Volitelně:

POST /api/download-job-item-fail.php



Ale na začátek to není nutné.

Důležitý detail: kdy přepnout na `downloaded`

Tady jsou dvě možnosti.

Varianta 1 – hned při začátku response

Jakmile server začne posílat soubor, označí ho jako `downloaded`.

Plus

- jednoduché

Mínus

- nevíte, zda se opravdu stáhl celý

Varianta 2 – až po potvrzení klientem

- server vydá jednorázový token
- browser soubor stáhne
- JS zavolá potvrzení `complete`

Plus

- formálně čistší

Mínus

- u klasického browser downloadu to stejně není úplně spolehlivé

Doporučení

Pro váš workflow bych použil:

- **označit jako** `downloaded` **ve chvíli, kdy server začne legálně vydávat soubor.**

Prakticky je to úplně stejné jako u jednotlivého downloadu dnes.

Jak spouštět downloady v browseru

Možnost 1 – hidden iframe

Každý soubor se stáhne přes dočasný skrytý iframe.

Výhody

- jednoduché
- funguje se standardním `Content-Disposition: attachment`

Nevýhody

- obtížněji poznáte přesný konec downloadu

Možnost 2 – `window.location` / `<a>`

Postupné generování a klikání na odkazy.

Výhody

- jednoduché

Nevýhody

- častěji naráží na blokace

Možnost 3 – `fetch` + `Blob`

To by normálně šlo, ale u velkých RAWů je to špatně:

- celé se to bufferuje v paměti
- nevhodné pro velké soubory

Takže `fetch` + `Blob` nedoporučuji.

Doporučení

Použít `iframe` nebo klasické `attachment URL`.

Návrh UI

Galerie

- checkboxy
- tlačítko **Stáhnout vybrané**

Stránka fronty

Například:

Bulk download
Připraveno: 32 souborů
Staženo: 11
Přeskočeno: 2
Zbývá: 19



Tlačítka:

- **Spustit stahování**
- **Pokračovat**
- **Zobrazit přeskočené**

A třeba seznam:

- IMG_1234.CR3 – downloaded
- IMG_1235.NEF – queued
- IMG_1236.ARW – skipped (locked by other user)

Co bych změnil proti ZIP návrhu

Už nepotřebujete

- tmpzip/
- ZipArchive
- cleanup starých ZIPů

Nově potřebujete

- tabulky `download_jobs` a `download_job_items`
- stránku s JS frontou
- endpoint pro jeden soubor
- endpoint pro status jobu

Praktické omezení, které je fér říct naplno

Pokud redaktor vybere třeba 150 RAWů, browser může:

- stáhnout prvních pár,
- pak zobrazit dotaz na povolení více stahování,
- nebo další automatiku zbrzdit.

Takže to není dobré chápat jako „server nařídí browseru 150 downloadů a ten poslušně vše udělá“. Reálně to bude:

- server připraví frontu,
- browser se pokusí stahovat po jednom,
- uživatel může jednorázově povolit více downloadů,
- případně občas klikne na pokračování.

To je normální.

Ještě lepší alternativy

A. Malý pomocný desktop klient

Nejčistší technické řešení, ale už je to mimo čisté PHP/browser.

B. WebDAV / síťové úložiště

Redaktoři by stahovali přes mounted storage. To je robustní, ale mění to celý provoz.

C. „Balíčky“ bez ZIPu na serveru přes speciální app

Také složitější.

Pro vaše zadání bych zůstal u **browser queue**.

Moje doporučení pro Sprint 3A

Ano, udělejme bulk download jako frontu jednotlivých souborů, ne jako ZIP.

Konkrétně:

- hromadný výběr v galerii
- vytvoření `download_job`
- atomické locknutí dostupných fotek
- stránka `/download-queue.php`
- JS, který postupně spouští downloady
- fallback tlačítko „Pokračovat“
- každá položka se po vydání souboru přepne na `downloaded`

To je podle mě pro press centrum lepší než ZIP.

Doporučené sprintové členění

Sprint 3A

- DB migrace pro `download_jobs`
- endpoint create job
- queue page
- single-file download endpoint
- základní progress UI

Sprint 3B

- lepší retry
- resume fronty po reloadu
- detailní chybové stavy
- admin dohled nad běžícími joby

Mohu teď rovnou připravit **upravený Sprint 3A jako přesnou technickou specifikaci**, včetně:

- SQL migrací,
- návrhu endpointů,
- stavového automatu,
- a kostry JS/PHP implementace.

Výborně, ale hned pokus o vytvoření tabulky `download_jobs` hlásí chybu:

ERROR 1005 (HY000): Can't create table

`press .download_jobs` (errno: 150 "Foreign key constraint is incorrectly formed")

Chyba `errno: 150 "Foreign key constraint is incorrectly formed"` u MariaDB téměř vždy znamená, že **cizí klíč neodpovídá přesně cílovému sloupci**.

Ve vašem případě je problém skoro jistě tady:

```
user_id INT NOT NULL,
```

```
</> SQL
```



a v tabulce `users` máte `id` nejspíš definované trochu jinak, než je `download_jobs.user_id`.

Nejčastější příčiny

Musí sedět všechno toto:

- stejný datový typ
- stejná signed/unsigned varianta
- cílový sloupec musí být indexovaný (PRIMARY KEY nebo UNIQUE)
- obě tabulky musí být InnoDB
- v odkazované tabulce i nové tabulce nesmí být rozdíl v definici, který MariaDB vyhodnotí jako nekompatibilní

Nejčastější praktický problém bývá:

- `users.id` je `INT UNSIGNED`
- ale `download_jobs.user_id` je jen `INT`

To pak spadne přesně takto.

Co udělat hned teď

Spustěte si:

```
</> SQL
```



a podívejte se přesně na definici `id`.

Typicky uvidíte něco jako:

```
</> SQL
```



Pak musí být `download_jobs.user_id` také:

```
</> SQL
```



Opravená verze tabulky

Pokud je `users.id` opravdu `INT UNSIGNED`, použijte:

```
CREATE TABLE download_jobs (  
  id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,  
  user_id INT UNSIGNED NOT NULL,
```

```
status ENUM('prepared','running','finished','failed') NOT NULL DEFAULT 'prepared',
created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
started_at DATETIME NULL,
finished_at DATETIME NULL,
CONSTRAINT fk_download_jobs_user
    FOREIGN KEY (user_id) REFERENCES users(id)
```

SQL



A stejně tak navazující tabulku:

```
CREATE TABLE download_job_items (
    id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    job_id BIGINT UNSIGNED NOT NULL,
    photo_id BIGINT UNSIGNED NOT NULL,
    seq_no INT NOT NULL,
    status ENUM('queued','downloading','downloaded','skipped','failed') NOT NULL,
    reason VARCHAR(100) NULL,
    locked_at DATETIME NULL,
    downloaded_at DATETIME NULL,
    CONSTRAINT fk_download_job_items_job
        FOREIGN KEY (job_id) REFERENCES download_jobs(id)
        ON DELETE CASCADE,
    CONSTRAINT fk_download_job_items_photo
        FOREIGN KEY (photo_id) REFERENCES photos(id)
        ON DELETE CASCADE,
    UNIQUE KEY uniq_job_seq (job_id, seq_no).
```

SQL



Jak zjistit přesný důvod chyby

Hned po neúspěšném `CREATE TABLE` spustíte:

SQL



A v části `LATEST FOREIGN KEY ERROR` MariaDB obvykle napíše přesněji, co nesedí.

Doporučená kontrola všech tabulek

Pro jistotu si zobrazte definice všech tří tabulek:


```
SHOW CREATE TABLE users;  
SHOW CREATE TABLE photos;
```

SQL



Zajímají vás hlavně:

- ENGINE=InnoDB
- typy id
- UNSIGNED

Nejpravděpodobnější oprava u vás

Protože dříve jsme měli návrh:

```
CREATE TABLE users (  
  id INT AUTO_INCREMENT PRIMARY KEY,
```

SQL



ale reálná databáze už může být jiná, nejpravděpodobnější je:

- vaše skutečné `users.id` není přesně `INT`
- nebo je `UNSIGNED`
- nebo tabulka `users` není `InnoDB`

Bezpečný postup

1. zjistit definici

SQL



2. podle ní upravit `download_jobs.user_id`

Například když uvidíte:

SQL



pak musí být:

SQL



Rychlá kontrolní poučka

Pro foreign key musí platit zhruba:

SQL



ne jen „podobný typ“, ale opravdu kompatibilní definice.

Pošlete sem výstup `SHOW CREATE TABLE users;` a případně `SHOW CREATE TABLE photos;` a já vám z toho rovnou složím přesné funkční `CREATE TABLE` příkazy pro vaše schéma.

ano, použil jsem nové návrhy tabulek a už to funguje. Co tedy teď dále?

Jak se dá z databáze vydumpovat schema? Udělal bych to a nahrál sem do projektu, abys viděl přesnou strukturu tabulek.

Ano, to je teď nejlepší další krok. Když sem nahraješ **přesné schéma databáze**, můžu už další části Sprintu 3A navrhovat přesně podle reality a ne podle odhadů.

Jak vydumpovat jen schéma databáze

Pro MariaDB/MySQL použij:

Bash



Kde:

- `--no-data` znamená jen struktura tabulek, bez dat
- `press` je název databáze
- výstup se uloží do souboru `press_schema.sql`

Pak ten soubor nahraj sem.

Když chceš i routines a trigger

Jestli bys chtěl opravdu úplnější strukturu:

↔ Bash



Když chceš jen konkrétní tabulky

Například:

↔ Bash



To je někdy přehlednější.

Alternativa přímo z MariaDB klienta

Když nechceš dělat dump celé DB, můžeš si vypsat definice jednotlivých tabulek:

```
SHOW CREATE TABLE users;  
SHOW CREATE TABLE photos;
```

↔ SQL



To ale bývá horší na ruční kopírování. Soubor z `mysqldump --no-data` je pohodlnější.

Pozor na citlivé údaje

Schéma obvykle nevdí sdílet, ale před nahráním se podívej, jestli dump neobsahuje něco navíc, co nechceš posílat. Typicky je to bezpečné, protože `--no-data` nevypisuje obsah tabulek.

Co uděláme potom

Jakmile nahraješ schema, navrhnu už přesně na míru:

- finální strukturu bulk download workflow,

- potřebné migrace, pokud ještě něco chybí,
- PHP endpoint `bulk-download-create.php`,
- endpoint pro jednotlivý download,
- a stránku fronty s JS logikou.

Nahraj sem `press_schema.sql` a navážeme rovnou konkrétní implementací.

 Soubor nahrán

Perfektní — díky, tohle je přesně to, co jsme potřebovali. Schéma je čisté a už obsahuje vše potřebné pro **download queue** (`download_jobs` + `download_job_items`).  `press_schema`

Můžeme rovnou implementovat **Sprint 3A – bulk download queue**.

Co už máš připravené (dobrá zpráva)

Z dumpu:

- `download_jobs` ✓
- `download_job_items` ✓
- `photos.status` (`ready`, `locked`, `downloaded`) ✓
- `photos.locked_by_user_id` ✓
- `photo_log` ✓

To znamená, že **DB** je připravená a nemusíme nic měnit.

Další krok: implementace backendu

Budeme potřebovat 4 endpointy:

1. `POST /bulk-download-create.php`
2. `GET /download-queue.php`
3. `GET /download-item.php`
4. `GET /api/download-job-status.php`

Implementujeme je postupně.

Krok 1 — vytvoření download jobů

Soubor:

bulk-download-create.php



Vstup:

POST photo_ids[]



Logika:

1. vytvořit download_jobs
2. pokusit se zamknout fotky
3. vložit do download_job_items
4. vrátit job_id

Implementace

```
<?php
declare(strict_types=1);

require __DIR__.'/bootstrap.php';

$userId = require_login();

$ids = $_POST['photo_ids'] ?? [];
$ids = array_values(array_unique(array_map('intval', $ids)));

if (!$ids) {
    http_response_code(400);
    exit('No photos selected');
}

$db->beginTransaction();

</> PHP
```



Krok 2 — stránka download fronty

Soubor:

download-queue.php



Tato stránka:

- načte job
- zobrazí progress
- JS spouští downloady jeden po druhém

```
<?php
require __DIR__.'/bootstrap.php';

$userId = require_login();

$jobId = (int)($_GET['job'] ?? 0);

?>
<html>
<head>
<meta charset="utf-8">
<title>Bulk download</title>
</head>
<body>

<h2>Bulk download</h2>

</> PHP
```



Krok 3 — API status fronty

/api/download-job-status.php



```
<?php
require __DIR__.'/../bootstrap.php';

$userId = require_login();

$jobId = (int)$_GET['job'];

$stmt = $pdo->prepare("
SELECT
COUNT(*) total,
SUM(status='downloaded') downloaded
FROM download_job_items
WHERE job_id=?
");

$stmt->execute(['$jobId']);

</> PHP
```



Krok 4 — download jedné položky



```
<?php
require __DIR__.'./bootstrap.php';

$userId = require_login();

$jobId = (int)$_GET['job'];
$itemId = (int)$_GET['item'];

$stmt = $pdo->prepare("
SELECT
i.id,
p.filepath,
p.filename,
p.id photo_id
FROM download_job_items i
JOIN photos p ON p.id=i.photo_id

</> PHP
```



Jak to bude fungovat

Redaktor:

1. zaškrtně 30 fotek
2. klikne **Bulk download**
3. server vytvoří job
4. otevře /download-queue.php
5. JS spustí downloady po jednom
6. každý download:
 - přepne photo → downloaded
 - loguje do jobu
7. hotovo

Další krok (doporučený)

Teď přidat do galerie:

```
<form method="POST" action="/bulk-download-create.php">
```



checkbox:

```
<input type="checkbox" name="photo_ids[]" value="<?= $photo['id'] ?>">
```



tlačítko:

```
<button>Bulk download</button>
```



Co tím získáš

- žádný ZIP
- RAW jdou přímo
- download fronta
- atomické lockování
- audit
- progress
- resume po reloadu

Tohle je přesně workflow pro press centrum 📷

Chceš teď:

1. doplnit **logování** do `photo_log`
2. přidat **progress bar** UI
3. přidat **resume po reloadu**
4. přidat **auto-start bez tlačítka**

Řekni číslo a pokračujeme.

Zdroje

Soubory jsem vytvořil jen nerozumím tomu Další krok (doporučený) To mám teď někam přidávat nebo mi teprve řekneš kam přesně s tím?

Dobrá otázka — tohle je potřeba upřesnit přesně.

Ten „Další krok“ patří **do stránky galerie fotek**, kde už dnes zobrazuješ seznam `photos` (ready/locked atd.). Tam se přidají checkboxy a formulář.

Kam to přesně patří

Do stránky typu např.:


```
photos.php
gallery.php
index.php
ready.php
```



tedy tam, kde máš něco jako:

```
</> PHP
```



a renderuješ náhledy.

1. Obalit galerii formulářem

Úplně nahoře nad seznamem fotek:

```
</> PHP
```



A úplně dole pod seznamem:

```
</> PHP
```



2. Přidat checkbox ke každé fotce

Tam kde renderuješ jednu fotku:

Například máš něco jako:

```
<div class="photo">
```

```
</> PHP
```



Upravíš na:

```
<div class="photo">

    <input
        type="checkbox"
        name="photo_ids[]"
        value="<?= $photo['id'] ?>"
    >


```

`</>` PHP



```
</div>
```

To je celé.

Kompletní příklad

```
<form method="POST" action="/bulk-download-create.php">

<div class="gallery">

<?php foreach ($photos as $photo): ?>

<div class="photo">

    <label>

        <input
            type="checkbox"
            name="photo_ids[]"
            value="<?= $photo['id'] ?>"
        >

    
```

`</>` PHP



Doporučuji přidat "vybrat vše"

Nad galerii:

```
<label>
<input type="checkbox" onclick="toggleAll(this)">
vybrat vše
</label>
```

```
<script>
function toggleAll(cb){
    document.querySelectorAll(
        'input[name="photo_ids[]"]'
    ).forEach(x => x.checked = cb.checked);
}
```

HTML



To je velmi praktické pro redaktory.

Co se stane po kliknutí

Po kliknutí na **Stáhnout vybrané**:

browser odešle:

```
POST /bulk-download-create.php
photo_ids[]=12
photo_ids[]=13
photo_ids[]=14
```



server vytvoří job a vrátí:

```
{
  "job_id": 7,
```

JSON



Tady je jedna důležitá věc:

Protože formulář očekává **HTML redirect**, upravíme `bulk-download-create.php`, aby dělal redirect místo JSON.

Na konci souboru změň:

```
echo json_encode([
    'job_id' => $jobId,
```

PHP



na:

To je důležité.

Výsledné flow

1. redaktor zaškrtně fotky
2. klikne "Stáhnout vybrané"
3. POST → `bulk-download-create.php`
4. server vytvoří job
5. redirect → `download-queue.php`
6. stránka začne stahovat soubory

Hotovo.

Pokud mi pošleš soubor galerie (třeba `photos.php`), upravím ti ho přesně na tvůj layout.

V tomhle se teď moc nevyznám. Asi by bylo lepší, kdybych do projektu uložil současný stav souborů na webu abys mi mohl přesněji říkat, co kam uložit ne? Mám ti zazipovat a poslat celou strukturu webu?

Ano — **tohle je přesně nejlepší další krok**. Pošli **celou strukturu webu** a já ti řeknu **přesně: do kterého souboru, na který řádek to vložit**. 👍

Co přesně poslat

Zazipuj **celý webroot projektu**, typicky něco jako:

```
press/  
├─ public/  
├─ templates/  
├─ includes/  
├─ api/  
├─ bootstrap.php  
├─ config.php  
├─ photos.php  
├─ index.php  
└─ ...
```



Stačí opravdu celý projekt (bez velkých upload adresářů).

Co naopak NEposílat (aby zip nebyl obří)

Můžeš vynechat:

- /var/presscentrum/original
- /var/presscentrum/preview
- /uploads
- /cache
- /node_modules
- /vendor (pokud je velké)

Ale když to pošleš i s nimi, nevadí.

Ideální obsah ZIPu

press-web.zip



uvnitř:

- PHP soubory
- HTML
- JS
- CSS
- adresářová struktura

To stačí.

Co udělám po nahrání

Jakmile to pošleš:

1. najdu stránku galerie
2. najdu render fotek
3. doplním checkboxy
4. doplním form
5. přidám bulk button
6. případně upravím routing
7. napíšu přesné diffy

Tj. dostaneš:

- "otevři soubor X"
- "za řádek Y vlož"
- "nahraď blok Z"

Takže žádné hádání.

Pošli ZIP a pokračujeme.

Ano — teď už to vidím přesně. Checkboxy a bulk download musíme přidat do `photos.php`, konkrétně kolem `<div class="photo-grid">`.

Níže máš přesné kroky co kam vložit.

1. Otevři soubor

web/photos.php



Najdi část:

```
<?php if (empty($photos)): ?>
<p>Žádné fotografie.</p>
<?php else: ?>
```

PHP



2. NAD `<div class="photo-grid">` vlož FORM

Uprav to takto:

```
<?php if (empty($photos)): ?>
<p>Žádné fotografie.</p>
<?php else: ?>

<form method="POST" action="/bulk-download-create.php">

<div class="bulk-toolbar">
<label>
<input type="checkbox" onclick="toggleAll(this)">
vybrat vše
</label>

<button type="submit">
Bulk download
</button>
```

3. Přidej checkbox do každé fotky

Najdi blok:

</> PHP



Změň ho na:

```
<div class="photo-card <?= $p['locked_by_user_id'] ? 'selected' : '' ?>">

<?php if ($p['status'] === 'ready' || ($p['status'] === 'locked' && $p['locked_by
<input
    class="bulk-checkbox"
    type="checkbox"
    name="photo_ids[]"
    value="<?= (int)$p['id'] ?>"
    onclick="event.stopPropagation();"
/>
```

</> PHP



Tohle zajistí:

- checkbox jen u ready
- checkbox i u vlastních locked
- ne u cizích locked
- ne u downloaded

4. Zavři formulář

Najdi konec gridu:

</div>

</> PHP



Uprav na:

```
</div>
```

```
</form>
```

```
</> PHP
```



5. Přidej JS na konec souboru

Těsně před:

```
</> PHP
```



vložit:

```
<script>
function toggleAll(cb) {
    document.querySelectorAll('.bulk-checkbox')
        .forEach(x => x.checked = cb.checked);
}
```

```
</> HTML
```



6. (volitelné) CSS

Do `assets/style.css` přidej:

```
.bulk-toolbar {
    margin-bottom: 10px;
    display: flex;
    gap: 15px;
    align-items: center;
}
```

```
.bulk-checkbox {
    position: absolute;
    top: 8px;
    left: 8px;
}
```

```
</> CSS
```



A ujisti se že `.photo-card` má:

```
.photo-card {
```

```
</> CSS
```



Co se teď stane

Na stránce:

- objeví se checkboxy
- "vybrat vše"
- tlačítko "Bulk download"

Kliknutí:

```
POST /bulk-download-create.php
```



→ redirect

→ `download-queue.php`

→ začne stahování

Další krok

Ted' potřebujeme jen upravit:

```
bulk-download-create.php
```

aby dělal redirect místo JSON.

Pošli mi jeho aktuální obsah a upravím ho přesně.

```
vše upraveno, jen nerozumím kroku 4  
</div>
```

```
<?php endif; ?>  
se v souboru mockrát opakuje a nevím, kam přesně  
navržený kód vložit
```

Díky — teď už vidím přesně tvůj soubor.

Formulář musí obalit **jen photo-grid**, tedy **přesně tento blok**:

```
<?php if (empty($photos)): ?>
<p>Žádné fotografie.</p>
<?php else: ?>

<div class="photo-grid">
...
</div>

</> PHP
```



Takže form začíná po `else:` a končí těsně před `<?php endif; ?>`.

Přesně co změnit

Najdi tento kód

```
<?php if (empty($photos)): ?>
<p>Žádné fotografie.</p>
<?php else: ?>

</> PHP
```



Změň ho na

```
<?php if (empty($photos)): ?>
<p>Žádné fotografie.</p>
<?php else: ?>

<form method="POST" action="/bulk-download-create.php">

<div class="bulk-toolbar">
<label>
<input type="checkbox" onclick="toggleAll(this)">
vybrat vše
</label>

<button type="submit">
Bulk download
</button>
```

Pak najdi konec gridu

V tvém souboru to je tady:

```
<?php endforeach; ?>
```

```
</div>
```

</> PHP



A změň na

```
<?php endforeach; ?>
```

```
</div>
```

```
</form>
```

</> PHP



Proč právě tady

Struktura teď bude:

```
if empty
else
  form
    toolbar
    photo-grid
      foreach photo
    end photo-grid
  end form
endif
```



Takže formulář obalí všechny checkboxy.

Jak si to ověřit

Po reloadu stránky:

- nad galerií uvidíš "vybrat vše"
- checkboxy na fotkách
- tlačítko "Bulk download"

To znamená, že je form správně.

Když to budeš mít, napiš.

Další krok: **bulk-download-create redirect fix**, jinak se nic nestane po kliknutí.

Ano, je to hotové, ale pokus o klik na bulk download vede na chybu:

```
[Tue Mar 31 12:04:29.571165 2026] [php:error] [pid 9861:tid 9861] [client 192.168.50.1:22325] PHP Fatal error: Uncaught Error: Failed opening required '/var/www/press/web/bootstrap.php' (include_path='.:usr/share/php') in /var/www/press/web/bulk-download-create.php:4\nStack trace:\n#0 {main}\n thrown in /var/www/press/web/bulk-download-create.php on line 4, referer: https://press.clovekavira.cz/photos.php
```

Přemýšlení po dobu 25s >

Ano — chyba je přesně v tom, že jsem v návrhu počítal se souborem `bootstrap.php`, ale ve tvém projektu **žádný** `bootstrap.php` **není**.

Tvůj projekt používá tuto strukturu:

- `config/config.php`
- `inc/db.php`
- `inc/auth.php`
- `inc/functions.php`

To je vidět i z `photos.php`. Schéma i struktura webu tomu odpovídají.  `press_schema`

Takže teď je potřeba upravit nové soubory, aby používaly stejný bootstrap jako zbytek aplikace.

Co opravit hned teď

1. `bulk-download-create.php`

Na začátku souboru nejspíš máš něco jako:

```
require __DIR__.'./bootstrap.php';
```

⌌ PHP



To je špatně.

Nahrad' začátek souboru tímto:

```
<?php
declare(strict_types=1);

require_once __DIR__ . '/config/config.php';
require_once __DIR__ . '/inc/auth.php';
require_once __DIR__ . '/inc/functions.php';
require_once __DIR__ . '/inc/db.php';
```

```
require_login();
```

```
$user = current_user();
```

⌌ PHP



Důležitá poznámka

U tebe `require_login()` **nic nevrací**, jen ověří přihlášení.

Takže správně musí být:

```
require_login();
```

⌌ PHP



a ne:

⌌ PHP



2. download-queue.php

Na začátku dej:

```
<?php
declare(strict_types=1);

require_once __DIR__ . '/config/config.php';
require_once __DIR__ . '/inc/auth.php';
require_once __DIR__ . '/inc/functions.php';
require_once __DIR__ . '/inc/db.php';

require_login();

$user = current_user();
$userId = (int)$user['id'];
$pdo = db();
```

⌌ PHP



3. api/download-job-status.php

Tady je cesta jiná, protože soubor leží v /api/ .

Na začátek dej:

```
<?php
declare(strict_types=1);

require_once __DIR__ . '/../config/config.php';
require_once __DIR__ . '/../inc/auth.php';
require_once __DIR__ . '/../inc/functions.php';
require_once __DIR__ . '/../inc/db.php';

require_login();

$user = current_user();
$userId = (int)$user['id'];
$pdo = db();
```

⌌ PHP



4. download-item.php

Na začátek dej:

```
<?php
declare(strict_types=1);

require_once __DIR__ . '/config/config.php';
require_once __DIR__ . '/inc/auth.php';
require_once __DIR__ . '/inc/functions.php';
require_once __DIR__ . '/inc/db.php';

require_login();

$user = current_user();
```

⌄ PHP 

Ještě jedna důležitá oprava navíc

V původním návrhu jsem používal `require_login()` jako by vracelo ID uživatele. Ve tvém projektu ale nevrací nic. To znamená, že ve všech těchto souborech musíš mít tento vzor:

```
require_login();

$user = current_user();
```

⌄ PHP 

To je teď standard pro celý projekt.

Co udělat nejrychleji

Ted' bych doporučil tento postup:

1. v `bulk-download-create.php` opravit `include` a `userId`
2. totéž v `download-queue.php`
3. totéž v `download-item.php`
4. totéž v `api/download-job-status.php`

Pak zkus znovu kliknout na **Bulk download**.

Co pravděpodobně přijde jako další chyba

Po opravě include se může objevit další problém, pokud `bulk-download-create.php` na konci stále vrací JSON místo redirectu.

Konec souboru má být takto:

PHP



```
a ne echo json_encode(...);
```

Nejkratší praktické řešení

Jestli chceš postup bez hledání, udělej teď jen toto:

V `bulk-download-create.php`

nahraď první řádky tímto:

```
<?php
declare(strict_types=1);

require_once __DIR__ . '/config/config.php';
require_once __DIR__ . '/inc/auth.php';
require_once __DIR__ . '/inc/functions.php';
require_once __DIR__ . '/inc/db.php';

require_login();

$user = current_user();
```

PHP



A pak zkus znovu kliknout.

Pošli případně nový error z logu a navážu přesně dalším fixem.

Zdroje

Nenašel jsem soubor `download-item.php`, jinak jsem změny provedl, ale máme chybu:
[Tue Mar 31 12:11:45.969597 2026] [php:error] [pid 8921:tid 8921] [client 192.168.50.1:11231] PHP Parse error:

syntax error, unexpected token "<", expecting end of file in
/var/www/press/web/bulk-download-create.php on line 4,
referer: https://press.clovekavira.cz/photos.php

Tohle je dobrá stopa. Ta chyba znamená, že v `bulk-download-create.php` je na začátku souboru **rozbitý PHP blok** — typicky takto:

```
<?php  
declare(strict_types=1);
```

PHP



nebo tam zůstal nějaký znak / HTML mimo PHP.

Co je špatně

Chyba:

syntax error, unexpected token "<", expecting end of file



na řádce 4 skoro vždy znamená, že se v souboru objevil **druhý** `<?php` **uvnitř už otevřeného PHP bloku**.

Jak to opravit

Otevři `bulk-download-create.php` a **celý začátek souboru** nastav přesně takto:

```
<?php  
declare(strict_types=1);  
  
require_once __DIR__ . '/config/config.php';  
require_once __DIR__ . '/inc/auth.php';  
require_once __DIR__ . '/inc/functions.php';  
require_once __DIR__ . '/inc/db.php';  
  
require_login();  
  
$user = current_user();
```

PHP



Důležité:

- nahoře má být **jen jedno** `<?php`
- nikde mezi tím už žádné další `<?php`
- před prvním `<?php` nesmí být žádný HTML text

Co zkontrolovat hned

Na úplném začátku souboru musí být něco jako:

```
<?php
```



a pak rovnou `require_once ...`

Nesmí tam být například:

```
<?php
declare(strict_types=1);
?>
```

```
<?php
```



ani

```
<?php
declare(strict_types=1);
```

```
<?php
```



K tomu `download-item.php`

To je v pořádku — ten soubor jsi ještě zřejmě nevytvořil.

Ted' se soustředíme jen na to, aby prošel `bulk-download-create.php`.

Nejrychlejší postup

Uděláš toto:

1. otevři `bulk-download-create.php`
2. smaž úplně první řádky
3. vlož tam tento blok:

```
<?php
declare(strict_types=1);

require_once __DIR__ . '/config/config.php';
require_once __DIR__ . '/inc/auth.php';
require_once __DIR__ . '/inc/functions.php';
require_once __DIR__ . '/inc/db.php';

require_login();

$user = current_user();
```

PHP



4. ulož
5. zkus znovu kliknout na **Bulk download**

Co čekám, že přijde potom

Až parse error zmizí, velmi pravděpodobně narazíme na další věc:

- buď chybí `download-queue.php`
- nebo `bulk-download-create.php` ještě vrací JSON místo redirectu
- nebo chybí `download-item.php`

To už je normální a půjdeme krok po kroku.

Pošli kdyžtak celý obsah `bulk-download-create.php` a já ti ho přepíšu přesně do funkční verze.

Už to funguje, ale i když vyberu fotky a pak dám Bulk download, napíše to jen No photos selected

To je už hodně dobrý posun. „No photos selected“ znamená, že `bulk-download-create.php` běží, ale do `$_POST['photo_ids']` se nic neposlalo.

Nejčastější příčina je jedna z těchto:

1. checkboxy nejsou uvnitř správného `<form>`
2. checkboxy nemají přesně jméno `photo_ids[]`
3. tlačítko odesílá jiný formulář než ten s checkboxy
4. v kartě fotky je ještě další `<form>` , který rozbije HTML strukturu

A u tebe je **čtvrtá možnost velmi pravděpodobná** — protože na kartách fotek už nejspíš máš vlastní formuláře pro lock/unlock/download, a HTML **nesmí vnořovat formulář do formuláře**.

Co se typicky děje

Ty jsi obalil galerii takto:

```
<form method="POST" action="/bulk-download-create.php">
  <div class="photo-grid">
    ... karta fotky ...
    <form method="post" action="lock.php">...</form>
    ...
```

⌕ PHP



To je neplatné HTML. Browser si to „opraví“ po svém a výsledkem často je, že checkboxy do bulk formuláře vůbec nepatří. Pak server správně řekne `No photos selected` .

Jak to rychle ověřit

Dočasně si do `bulk-download-create.php` hned za načtení přidej:

```
echo '<pre>';
var_dump($_POST);
```

⌕ PHP



Když po kliknutí uvidíš něco jako:

⌕ PHP



nebo chybí `photo_ids` , problém je opravdu ve formuláři na stránce galerie.

Správné řešení

Bulk download formulář **nesmí obalovat celou galerii**, pokud uvnitř karet už jsou jiné formuláře.

Místo toho uděláme:

- checkboxy necháme u fotek
- bulk tlačítko dáme do **samostatného formuláře**
- a před odesláním JavaScript zkopíruje zaškrtnuté ID do hidden inputů

To funguje spolehlivě i když jsou v kartách další formuláře.

Co přesně změnit

1. V `photos.php` ZRUŠ obalující form kolem galerie

Tedy smaž:

```
</> PHP
```



a také smaž uzavírací:

```
</> PHP
```



kolem `.photo-grid`.

Galerie má zůstat bez společného outer formu.

2. Nahoře nad galerií dej samostatný bulk formulář

Vlož nad `<div class="photo-grid">` toto:

```
<div class="bulk-toolbar">
  <label>
    <input type="checkbox" onclick="toggleAll(this)">
    vybrat vše
  </label>

  <form id="bulk-download-form" method="POST" action="/bulk-download-create.php">
    <button type="submit">Bulk download</button>
```

```
</> PHP
```



</> PHP

</div>



Důležité: ten formulář je prázdný, checkboxy v něm nejsou.

3. Checkboxy u fotek nech takto

U každé fotky:

```
<?php if ($p['status'] === 'ready' || ($p['status'] === 'locked' && $p['locked_by']  
<input  
    class="bulk-checkbox"  
    type="checkbox"  
    name="photo_ids[]"  
    value="<?= (int)$p['id'] ?>"  
    onclick="event.stopPropagation();"
/>
```

</> PHP



To je v pořádku.

4. Přidej JS, který před odesláním vyrobí hidden inputy

Těsně před footer vlož toto:

```
<script>  
function toggleAll(cb) {  
    document.querySelectorAll('.bulk-checkbox').forEach(x => {  
        x.checked = cb.checked;  
    });  
}  
  
document.getElementById('bulk-download-form').addEventListener('submit', function() {  
    const form = this;  
  
    form.querySelectorAll('input[name="photo_ids[]"]').forEach(el => el.removeAttribute('checked'));  
  
    const checked = document.querySelectorAll('.bulk-checkbox:checked');  
  
    if (checked.length === 0) {  
        e.preventDefault();  
    }  
});
```

</> HTML



Tohle udělá přesně to, co potřebujeme:

- posbírání zaškrtnuté checkboxy z galerie
- vloží jejich ID do bulk formuláře
- odešle je na server

Proč je to lepší

Protože:

- nemusíš řešit vnořené formuláře
- můžeš si nechat individuální formuláře na kartách
- bulk download bude fungovat vedle nich

Co bych udělal hned teď

1. zrušit `<form ... bulk-download-create.php>` kolem celé galerie
2. dát nahoře samostatný formulář jen s tlačítkem
3. přidat ten JS blok

Pak `No photos selected` zmizí.

Ještě jedna rychlá kontrola

V `bulk-download-create.php` můžeš mít validaci takto:

```
$ids = $_POST['photo_ids'] ?? [];  
$ids = array_values(array_unique(array_map('intval', $ids)));  
  
if (!$ids) {  
    http_response_code(400);  
}
```

PHP



To je v pořádku. Chyba je téměř jistě na straně HTML formuláře, ne v PHP.

Pošli klidně část `photos.php` kolem `.photo-grid` a upravím ti ji přesně do finální podoby.

 Soubor nahrán

Ano prosím uprav to. Vybrat vše a Bulk download se teď na stránce objevuje dvakrát pod sebou.

Ano — vidím přesně problém. V souboru máš teď **dva bulk toolbary za sebou**: jeden správný se samostatným `<form id="bulk-download-form">` a hned pod ním druhý starý blok bez formuláře. To je přímo v nahraném `photos.php`.  `photos`

Uděláme to jednoduše.

Co smazat

V `photos.php` smaž celý tento druhý blok:

```
<div class="bulk-toolbar">
  <label>
    <input type="checkbox" onclick="toggleAll(this)">
    vybrat vše
  </label>

  <button type="submit">
    Bulk download
```

 PHP



Ten je navíc a způsobuje duplicitní zobrazení.

Jak má správně vypadat začátek bulk části

V souboru má po úpravě zůstat jen jeden toolbar, a to přesně takto:

```
<?php if (empty($photos)): ?>
<p>Žádné fotografie.</p>
<?php else: ?>

<div class="bulk-toolbar">
  <label>
    <input type="checkbox" onclick="toggleAll(this)">
    vybrat vše
  </label>

  <form id="bulk-download-form" method="POST" action="/bulk-download-create.php">
    <button type="submit">Bulk download</button>
  </form>
</div>
```


</> PHP



Tohle je správná verze. Odpovídá tomu, co už v souboru jednou máš.  photos

Co má následovat dál

Hned po tom má být:

</> PHP



a pak jednotlivé `.photo-card`.

Co tedy udělat krok po kroku

1. Otevři `photos.php`
2. Najdi první `<div class="bulk-toolbar">` — ten **ponech**
3. Najdi hned pod ním druhý `<div class="bulk-toolbar">` — ten **celý smaž**
4. Ulož soubor
5. Reload stránky

Jak má vypadat výsledná struktura

Správně to má být takto:

```
<?php if (empty($photos)): ?>
<p>Žádné fotografie.</p>
<?php else: ?>

<div class="bulk-toolbar">
  <label>
    <input type="checkbox" onclick="toggleAll(this)">
    vybrat vše
  </label>

  <form id="bulk-download-form" method="POST" action="/bulk-download-create.php">
    <button type="submit">Bulk download</button>
  </form>
</div>

<div class="photo-grid">
```

</> PHP



Až to smažeš, zkus znovu vybrat pár fotek a kliknout na **Bulk download**. Pak už by se mělo přejít na `bulk-download-create.php` se správně odeslanými `photo_ids[]`.

Zdroje

Pošli mi prosím kompletní celé nové znění souboru `photos.php`

Jasně — tady je kompletní nové znění `photos.php`, už opravené na jednu bulk lištu a se samostatným bulk formulářem. Vychází z tebou nahraného souboru.  `photos`

```
<?php
declare(strict_types=1);

require_once __DIR__ . '/config/config.php';
require_once __DIR__ . '/inc/auth.php';
require_once __DIR__ . '/inc/functions.php';
require_once __DIR__ . '/inc/photos.php';

require_login();

if (!has_permission('photos.view')) {
    http_response_code(403);
    exit('Přístup odepřen.');
}

$ftnUser = isset($_GET['ftn_user']) ? trim((string)$_GET['ftn_user']) : '';
```

PHP

Po uložení zkus vybrat pár fotek a kliknout na **Bulk download**. Další logický krok bude doladit `bulk-download-create.php`, `download-queue.php` a vytvořit chybějící `download-item.php`.

Zdroje

Aha, rozumím, tys přidal do fotek zaškrtnutí pro výběr. Ale to je špatně. Dohodli jsem někde v průběhu Sprintu 2 nebo 3, že na výběr bude sloužit funkce lock aby ostatní redaktoři viděli, které fotky jsou zamčené a někdo je bude nahrávat. Zaškrtnutí zruš a funkci bulk download uprav tak, aby zahrnula fotky ve stavu locked.

Jasně. Máš pravdu — checkboxy jsou v rozporu s už domluveným workflow. Podle schématu už na to máš správná pole: `photos.status` a `photos.locked_by_user_id`, takže **bulk download** má brát automaticky fotky, které jsou `locked` aktuálním uživatelem.

 `press_schema`

To znamená:

- žádná zaškrtnutí
- v `photos.php` zůstane jen tlačítko **Bulk download locked**
- `bulk-download-create.php` už nebude číst `$_POST['photo_ids']`
- místo toho si z DB načte:
 - `status = 'locked'`
 - `locked_by_user_id = aktuální uživatel`

Níže máš hotové nové `photos.php`.

Nové celé znění `photos.php`

Vychází z tvého nahraného souboru, jen jsem odstranil checkboxy a upravil bulk lištu.

 `photos`

```
<?php
declare(strict_types=1);

require_once __DIR__ . '/config/config.php';
require_once __DIR__ . '/inc/auth.php';
require_once __DIR__ . '/inc/functions.php';
require_once __DIR__ . '/inc/photos.php';

require_login();

if (!has_permission('photos.view')) {
    http_response_code(403);
    exit('Přístup odepřen.');
```

PHP



Co teď upravit v bulk-download-create.php

Tam je potřeba změnit logiku z:

```
$ids = $_POST['photo_ids'] ?? [];
```

PHP



na:

```
$stmt = $pdo->prepare("
    SELECT id
    FROM photos
    WHERE status = 'locked'
    AND locked_by_user_id = ?
    ORDER BY uploaded_at ASC, id ASC
");
$stmt->execute([$userId]);
$ids = array_map('intval', $stmt->fetchAll(PDO::FETCH_COLUMN));

if (!$ids) {
    http_response_code(400);
}
```

PHP



A zbytek create jobu může běžet nad těmi `id`.

Důležitá změna uvnitř bulk-download-create.php

Původně jsme počítali s tím, že se fotky teprve při bulk akci lockují.

Teď už ne. Teď se mají do jobu přidat **jen ty, které už jsou locked tímto uživatelem**.

Takže uvnitř smyčky má být podmínka zjednodušená na:

```
if (
    $photo['status'] === 'locked'
    && (int)$photo['locked_by_user_id'] === $userId
) {
    $status = 'queued';
} else {
```

PHP



```
}  
    $status = 'skipped',  
}
```

a žádný `UPDATE photos SET status='locked' ...` už tam nemá být.

Prakticky

Teď je správný workflow:

- redaktor klikáním zamyká fotky přes `ready -> locked`
- ostatní vidí, že jsou zabrané
- pak klikne na **Bulk download locked**
- systém vezme všechny jeho `locked`
- vytvoří download job
- jednotlivé soubory stáhne
- při vydání se přepnou do `downloaded`

To přesně odpovídá vašemu modelu.

Pošli mi teď obsah `bulk-download-create.php` a přepíšu ho rovnou do správné finální verze.

Zdroje

 Soubor nahrán

Super — tady je problém jasný. V souboru stále zůstala stará logika, která **se snaží znovu lockovat fotky**, i když už jsou locknuté. To je přesně ta část s `$lock` a `status='ready'`. To musíme úplně odstranit.  `bulk-download-create`

Navíc je potřeba změnit:

- nevracet JSON
- ale udělat redirect na `download-queue.php`
- odstranit `$lock` query úplně

Níže máš **kompletní finální verzi souboru**.

Nový `bulk-download-create.php`

Nahrad celý soubor tímto:

```
<?php  
declare(strict_types=1);
```

```
require_once __DIR__ . '/config/config.php';
require_once __DIR__ . '/inc/auth.php';
require_once __DIR__ . '/inc/functions.php';
require_once __DIR__ . '/inc/db.php';

require_login();

$user = current_user();
$userId = (int)$user['id'];
$pdo = db();

/*
 * ...
 */

</> PHP
```

Co se teď bude dít

Po kliknutí **Bulk download locked:**

1. vybere všechny:

```
status = locked
locked_by_user_id = aktuální user
```

2. vytvoří download_jobs
3. vloží položky do download_job_items
4. redirect:

```
/download-queue.php?job=XX
```

5. queue začne stahovat soubory

Co jsme odstranili

Úplně pryč je:

- \$lock UPDATE
- kontrola status='ready'
- JSON response
- duplicitní logika

Teď je to přesně:

```
ready -> locked (ručně)
locked -> bulk download
download -> downloaded
```

Což odpovídá vašemu workflow.

Další krok

Ted' pravděpodobně narazíme na:

- chybějící `download-queue.php`
- nebo chybějící `download-item.php`

To je v pořádku — to je další krok Sprint 3A.

Zdroje

Výborně, teď je to výrazně lepší. Počítadlo Moje zamčené na této stránce počítá správně a když se klikne na Bulk download Locked, objeví se jednoduchá stránka Bulk download. Ovšem po kliknutí na Start se v logu periodicky objevuje:

```
[Tue Mar 31 12:36:28.247645 2026] [php:error] [pid 9863:tid 9863] [client 192.168.50.1:48689] script '/var/www/press/web/download-item.php' not found or unable to stat, referer: https://press.clovekavira.cz/download-queue.php?job=2
```

Přesně tak — teď už chybí jen další stavební kámen: **soubor** `download-item.php` **zatím neexistuje**, ale `download-queue.php` se ho snaží volat. Proto Apache hlásí:

```
script '/var/www/press/web/download-item.php' not found
```



To je vlastně dobrá zpráva: fronta už běží až k samotnému vydání souboru.

Co teď udělat

Vytvoř nový soubor:

```
/var/www/press/web/download-item.php
```



a vlož do něj toto celé znění:

```
<?php
declare(strict_types=1);

require_once __DIR__ . '/config/config.php';
require_once __DIR__ . '/inc/auth.php';
require_once __DIR__ . '/inc/functions.php';
require_once __DIR__ . '/inc/db.php';
```

```
require_login();

$user = current_user();
$userId = (int)$user['id'];
$pdo = db();

$jobId = (int)($_GET['job'] ?? 0);
$itemId = (int)($_GET['item'] ?? 0);

</> PHP
```

Co tohle udělá

Ten soubor:

- ověří přihlášení
- ověří, že položka fronty patří aktuálnímu redaktorovi
- ověří, že fotka je pořád `locked` a zamčená tímto uživatelem
- označí položku fronty jako `downloaded`
- přepne fotku `locked` -> `downloaded`
- zapíše `photo_log`
- pošle originál do browseru

Co pak zkusit

1. vytvořit `download-item.php`
2. uložit
3. znovu otevřít `Bulk download`
4. kliknout na `Start`

Pak by se už měly začít opravdu stahovat jednotlivé soubory.

Pravděpodobný další krok

Až tohle bude fungovat, další věc ke kontrole bude, jestli se po stažení poslední položky správně ukončí job v `download_jobs` na `finished`. To se dá doplnit hned potom.

Perfektní! fotky se stáhly (prohlížeč si vyžádal povolení stažení několika souborů), ale všechny se postupně stáhly. Jejich stav se změnil na `downloaded`, což je přesně, jak to chceme.

Jen bude potřeba zpříjemnit UX při tom downloadu.

Možná namísto stránky s tlačítkem `start` jen nějaký vyskakovací dialog "Bude se stahovat 5 fotografií.

Pokračovat? Ano/ne". V průběhu downloadu něco jako "Probíhá download 3/5" třeba namísto "Moje zamčené na této stránce". Po downloadu by bylo dobré, aby se

refreshnula stránka galerie, aby se hned ukázaly změněné stavy fotek (teď jsem ji refreshoval ručně).

Skvělé — tohle už je velmi dobrý milník. Chování je teď funkčně správně:

- bulk download funguje,
- browser stáhne soubory postupně,
- stavy se přepnou na `downloaded`,
- workflow zůstává konzistentní.

A ano, teď dává smysl udělat **Sprint 3A UX polish**.

Co bych upravil

Souhlasím s tebou v plném rozsahu:

1. zrušit mezistránku se Start
2. před spuštěním ukázat potvrzovací dialog
3. během stahování zobrazit průběh přímo v galerii
4. po dokončení automaticky obnovit galerii

Doporučené chování

Po kliknutí na `Bulk download locked`

místo přechodu na samostatnou stránku:

- backend vytvoří job jako teď
- galerie se otevře znovu
- vyskočí dialog:

Bude se stahovat 5 fotografií. Pokračovat?
[Ano] [Ne]



Po potvrzení

na stránce galerie se zobrazí třeba lišta:

Probíhá download 3 / 5



a po dokončení:

Download dokončen. Obnovuji galerii...



pak automatický reload stránky.

To je podle mě nejlepší UX.

Nejjednodušší technické řešení

Nebudeme používat samostatnou `download-queue.php` jako uživatelskou stránku.

Místo toho:

- `bulk-download-create.php` vytvoří job
- pak redirectne zpět na `photos.php?download_job=123&download_total=5`
- `photos.php` podle parametru pozná, že má:
 - zobrazit potvrzovací dialog
 - po potvrzení spustit download frontu na pozadí
 - ukazovat průběh
 - po dokončení reloadnout stránku bez parametrů

Tím pádem:

- redaktor pořád zůstává v galerii
- vidí stejné fotky
- po dokončení se rovnou zobrazí aktualizované stavy

To je výrazně příjemnější.

Co konkrétně změnit

1. `bulk-download-create.php`

místo redirectu na:

```
</> PHP
```



bude redirect na:

```
</> PHP
```



tedy konec souboru změnit na:

```
header("Location: /photos.php?download_job=" . $jobId . "&download_total=" . cour
```

```
</> PHP
```



Ještě lépe přes skutečný počet vložených queued položek, ale pro první verzi stačí `count($ids)` .

2. photos.php

doplnit:

- načtení `download_job` a `download_total` z `$_GET`
 - malý overlay/dialog
 - JS, který po potvrzení začne volat downloady
 - průběhový text
 - po dokončení reload bez query parametrů
-

Jak by to vypadalo v praxi

Nahoře v `photos.php`

doplnit:

PHP



Do stránky přidat stavový box

třeba pod bulk toolbar:

```
<?php if ($downloadJobId > 0 && $downloadTotal > 0): ?>
<div id="download-progress" class="bulk-toolbar">
    Připraven download <?= $downloadTotal ?> fotografií.
```

PHP



Na konec `photos.php` přidat JS

logika:

- po načtení stránky:
 - `confirm("Bude se stahovat 5 fotografií. Pokračovat?")`

- pokud Ano:
 - začne se fronta
 - text se mění na Probíhá download X / Y
- po dokončení:
 - window.location = '/photos.php?...' bez download_job parametru
 - tím se galerie refreshne

Přesný návrh JS

Tahle verze je jednoduchá a dobře čitelná:

```
<script>
async function runBulkDownload(jobId, total) {
  let completed = 0;
  const progressBar = document.getElementById('download-progress');

  async function fetchStatus() {
    const r = await fetch('/api/download-job-status.php?job=' + jobId, { c:
    return await r.json();
  }

  async function step() {
    const status = await fetchStatus();

    completed = status.downloaded || 0;

    if (progressBar) {
```

HTML



Co to vyřeší

Tohle přesně splní tvoje požadavky:

- žádná mezistránka se Start
- potvrzení Ano/Ne
- průběh přímo v galerii
- po dokončení automatický refresh galerie

Co bych ještě lehce doladil

Místo textu „Moje zamčené na této stránce“

během downloadu může být nahrazen textem průběhu.

Tedy v `photos.php` :

- normálně:
 - Moje zamčené na této stránce: X
- pokud je aktivní `download_job` :
 - zobrazit Probíhá download 0 / N

To je přehledné.

Zrušit nebo nechat `download-queue.php`

- může zůstat technicky v projektu
- ale už nebude potřeba jako uživatelská stránka

Malé doporučení navíc

Až bude tohle hotové, další drobný upgrade by byl:

- po dokončení nastavit v `download_jobs` :
 - `status = 'finished'`
 - `finished_at = NOW()`

To už je spíš pořádek v datech, ne UX.

Co teď udělat prakticky

Teď bych doporučil tento přesný postup:

1. upravit `bulk-download-create.php` redirect na `photos.php?download_job=...`
2. doplnit do `photos.php` čtení `download_job` a `download_total`
3. přidat progress box
4. přidat JS s `confirm()` a `auto-refresh`

Pošli mi teď prosím znovu aktuální `photos.php` a `bulk-download-create.php` a já ti rovnou pošlu **celé finální znění obou souborů**, už přesně pro tenhle UX model.

📁 Soubor nahrán 📁 Soubor nahrán

Soubory přiloženy

Perfektní. Tady jsou **celá nová znění obou souborů** pro ten UX model, který jsi popsal:

- po kliknutí na **Bulk download locked** se nevstupuje na mezistránku,
- galerie se znovu otevře s parametry jobu,
- vyskočí potvrzení Bude se stahovat X fotografií. Pokračovat? ,
- během stahování se nahoře zobrazuje průběh,
- po dokončení se galerie sama obnoví bez parametrů,
- změněné stavy se hned ukážou.

Vycházím z tebou nahraných aktuálních verzí obou souborů.  bulk-download-create

 photos

bulk-download-create.php

Nahrad celý soubor tímto:

```
<?php
declare(strict_types=1);

require_once __DIR__ . '/config/config.php';
require_once __DIR__ . '/inc/auth.php';
require_once __DIR__ . '/inc/functions.php';
require_once __DIR__ . '/inc/db.php';

require_login();

$user = current_user();
$userId = (int)$user['id'];
$pdo = db();

/*
 * vezmeme všechny fotky locked tímto uživatelem
 */
```

  PHP

photos.php

Nahrad celý soubor tímto:

```
<?php
declare(strict_types=1);
```



```
require_once __DIR__ . '/config/config.php';
require_once __DIR__ . '/inc/auth.php';
require_once __DIR__ . '/inc/functions.php';
require_once __DIR__ . '/inc/photos.php';

require_login();

if (!has_permission('photos.view')) {
    http_response_code(403);
    exit('Přístup odepřen.');
```

</> PHP



Co se tím změní

Po kliknutí na **Bulk download locked**:

- backend vytvoří job
- vrátíš se zpět do galerie
- vyskočí potvrzení
- při potvrzení začne stahování
- nahoře se bude měnit text:
 - Probíhá download 0 / 5
 - Probíhá download 3 / 5
 - Download dokončen. Obnovuji galerii...
- galerie se sama reloadne
- stavy fotek už budou rovnou downloaded

Jedna drobná poznámka

Ted' se po redirectu resetují filtry a stránkování, protože `bulk-download-create.php` posílá uživatele jen na `/photos.php?... .` Kdybys chtěl, v dalším kroku můžeme zachovat i aktuální:

- `ftp_user`
- `status`
- `page`

aby se po dokončení galerie obnovila přesně tam, kde redaktor byl.

Super, funguje. Pár drobností: Namísto Bulk download locked dej do tlačítka nápis Hromadné stažení zamčených. A místo "Probíhá download 0/2" buďto dej "Downloadováno 0 z 2 nebo raději nech Probíhá download, ale ukaž počítadlo o jedničku zvýšené. "Probíhá download 1/2" lépe odpovídá skutečnému stavu, že download první fotografie právě probíhá. Ještě otázka bokem. POšli mi SQL příkaz, kterým teď vrátím všechny fotky ve stavu download do stavu ready (už jsem všechny downloadoval a nemám teď na čem zkoušet zaškrtávání locked.

Jasně 👍 Uděláme dvě drobné UX úpravy + SQL.

1. Text tlačítka

V `photos.php` změň:

⌕ PHP



na:

⌕ PHP



2. Počítadlo „Probíhá download 1/2“

Ted' se zobrazuje počet již dokončených.

Chceš zobrazovat právě probíhající, takže přičteme +1 (pokud existuje next_item).

V `photos.php` v JS změň blok:

```
const downloaded = Number(status.downloaded || 0);

if (statusBox) {

⌕ JavaScript
```



na:

```
const downloaded = Number(status.downloaded || 0);  
const current = status.next_item ? downloaded + 1 : downloaded;
```

```
if (statusBox) {
```

```
</> JavaScript
```



Tím dostaneš:

```
Probíhá download 1 / 2
```

```
Probíhá download 2 / 2
```

```
Download dokončen...
```



což odpovídá reálnému stavu.

3. SQL — vrátit downloaded → ready

Bezpečná varianta (jen status):

```
UPDATE photos
```

```
</> SQL
```



Doporučená varianta (vyčistit i metadata):

```
UPDATE photos
```

```
SET
```

```
  status = 'ready',
```

```
  downloaded = 0,
```

```
</> SQL
```



Pokud chceš zároveň zrušit locky:

```
UPDATE photos
```

```
SET
```

```
  status = 'ready',
```

```
  downloaded = 0,
```

```
downloaded_at = NULL,
```

```
</> SQL
```



To je ideální pro testování.

Bonus – reset i download queue (doporučeno pro testy)

Pokud chceš čistý stav:

```
</> SQL
```



Pak můžeš testovat znovu úplně od nuly.

Výborně, teď to funguje podle mých představ. Čím navrhuješ pokračovat?

Skvělé. V tom případě je **Sprint 3A funkčně hotový** a já bych pokračoval takto:

Nejlepší další krok: Sprint 3B – dotažení robustness a administrace

Teď už to funguje pro běžný provoz. Další největší hodnotu přinese, když systém začne být odolnější a lépe spravovatelný.

1. Dokončení jobů a úklid fronty

Teď bych doplnil:

- po dokončení všech položek nastavit `download_jobs.status = 'finished'`
- vyplnit `finished_at`
- při chybě nastavovat `failed`
- případně cron na mazání starých jobů

To je důležité kvůli pořádku v DB a pozdějšímu debugování.

2. Lepší stavování a logování

Doplnil bych:

- log akce `bulk_download_started`
- log dokončení jobu
- log chybějícího souboru nebo selhané položky

- možnost v adminu vidět, co kdo hromadně stáhl

To se bude hodit při ostrém provozu.

3. Stránka „Moje zamčené“

Tohle bych doporučil velmi silně.

Samostatná stránka nebo filtr:

- zobrazí jen moje `locked`
- nahoře tlačítko **Hromadné stažení zamčených**
- případně **Odemknout vše moje**

To redaktorům výrazně zpříjemní práci, protože nebudou hledat svoje zamčené kusy mezi všemi fotkami.

4. Zachování filtrů po bulk downloadu

Ted' se po dokončení galerie obnoví, ale je dobré zachovat:

- stránku
- filtr fotografa
- filtr stavu

To je malá úprava, ale UX zlepší dost.

5. Ošetření výjimek

Doplnil bych:

- co když soubor mezitím zmizí
- co když se download přeruší
- co když je položka už `downloaded`
- co když uživatel zavře panel během fronty

To jsou typické produkční hrany.

Pořadí, které bych doporučil

Varianta nejpraktičtější

1. Sprint 3B – dokončení a úklid download jobů
2. Stránka „Moje zamčené“
3. Admin dohled nad locky a downloady
4. UX drobnosti a filtry

Kdybych měl vybrat jedinou věc hned teď

Tak bych šel do:

„Moje zamčené“ + hromadné stažení přímo odtud

Protože to redaktoři opravdu pocítí při každodenní práci.

Typický workflow by pak byl:

- redaktor prochází fotky
- kliká ready -> locked
- otevře **Moje zamčené**
- zkontroluje výběr
- klikne **Hromadné stažení zamčených**

To je velmi čisté.

Alternativně: Sprint 4 – administrace

Pokud chceš teď spíš provozní jistotu než UX, tak další krok může být:

- přehled všech zamčených fotek
- kdo je má zamčené
- jak dlouho
- admin unlock
- přehled download jobů

To je výborné pro správu redakce.

Moje doporučení: **pokračujme Sprintem 3B: dokončení download jobů + stránka Moje zamčené.**

Tvoje návrhy se mi líbí, ale projekt už teď je hodně rozvinutý a potřeboval bych si pomalu sjednat nějaké testery (zatím jsme například nezkoušeli chování při více fotografech a více redaktorech). Asi by bylo nejlepší před dalším pokračování práce s fotkami teď otevřít kapitolu user managementu, abych mohl snadným způsobem vytvořit potřebné usery a povolat je k testování. Co si o tom myslíš?

Myslím, že je to teď **správný krok**.

Největší riziko už není „umíme ještě jednu funkci navíc“, ale spíš:

- jak se systém chová s více rolemi současně,
- jak snadno dostaneš do systému testery,
- jak snadno jim nastavíš práva,
- a jak snadno je po testování zase upraviš nebo vypneš.

Takže ano: **otevřel bych teď kapitolu user managementu dřív, než budeme dál rozvíjet workflow fotek.**

Proč to dává smysl právě teď

Už máš funkční základ:

- upload,
- preview,
- lock/unlock,
- single download,
- bulk download,
- stavový workflow.

To už je dost na to, aby to šlo **reálně testovat v malém provozu**. Jenže bez pohodlné správy uživatelů bude každé testování zbytečně bolestivé.

Místo dalšího vývoje fotek bych teď investoval do toho, aby šlo:

- založit nového fotografa,
- založit nového redaktora,
- přiřadit roli,
- nastavit FTP účet,
- aktivovat/deaktivovat účet,
- případně resetovat heslo.

To ti odemkne testování s více lidmi mnohem víc než další jedna funkce ve fotkách.

Co bych do user managementu zařadil jako MVP

Nešel bych zatím do ničeho přehnaně složitého. Na první použitelnou verzi stačí:

1. Seznam uživatelů

Tabulka:

- jméno
- login
- FTP user
- role
- aktivní/neaktivní
- poslední přihlášení

2. Vytvoření uživatele

Formulář:

- jméno
- příjmení
- web login
- heslo
- FTP login
- FTP heslo
- role
- homedir
- aktivní

3. Editace uživatele

- změna jména
- změna role
- změna aktivace
- změna FTP údajů
- volitelně změna hesla

4. Aktivace/deaktivace

To je důležité pro testery:

- nemusíš mazat účet
- jen ho vypneš

5. Role

Na začátek klidně stačí pár systémových rolí:

- admin
- editor
- photographer

Až později jemnější oprávnění.

Co bych zatím odložil

Na první sprint user managementu bych nedával:

- self-service registraci,
- e-mailové obnovování hesla,
- složitý audit každé změny oprávnění,
- import uživatelů z CSV,
- víceúrovňové delegování práv.

To může přijít později.

Praktický důvod: testovací scénáře

Jakmile budeš mít user management, můžeš konečně rozumně testovat scénáře typu:

- 2 fotografové nahrávají současně,
- 2 redaktoři zamykají stejné fotky,
- jeden redaktor lockne a druhý vidí blokaci,
- fotograf A vidí jen svoje,
- admin vidí vše,
- neaktivní účet se nepřihlásí,
- vypnutý FTP účet nepřijme upload.

To jsou přesně testy, které teď mají větší hodnotu než další nová funkce.

Moje doporučení pro další sprint

Sprint 4A – User management MVP

Cíl:

umožnit ti pohodlně zakládat a spravovat testovací uživatele.

Obsah:

- seznam uživatelů
- vytvoření uživatele
- editace uživatele
- aktivace/deaktivace
- role selector
- změna web hesla
- změna FTP hesla

Sprint 4B – Role a oprávnění

Až potom:

- jemnější mapování práv
- správa rolí přes UI
- případně přehled permissions

Sprint 4C – Testovací provoz

Pak:

- připravit testovací scénáře
- projít multi-user chování
- dohledat kolize a UX problémy

Jedna důležitá poznámka

Protože už v DB máš:

- `users`

- roles
- permissions
- role_permissions

tak bych teď neřešil návrh databáze od nuly, ale spíš se opřel o to, co už máš, a postavil nad tím administrační UI. To bude nejrychlejší.

Co bych udělal hned teď

Za mě další krok:

otevřít Sprint 4A – User management MVP

a začít tímto pořadím:

1. stránka seznamu uživatelů
2. formulář pro přidání uživatele
3. formulář pro editaci uživatele
4. aktivace/deaktivace
5. napojení na role

To ti velmi rychle umožní pozvat testery a rozběhnout reálné zkoušení.

Pošli klidně aktuální soubory kolem userů a adminu, nebo znovu webový balík, a navrhnu přesně první implementační krok Sprintu 4A.

Ano, strukturu databáze jsi mi navrhnul už někdy na začátku projektu, je založená a v tabulce "role" jsou role superadmin, admin, press_operator a photographer, a toho bych se držel. User management bude vždy velmi jednoduchý, superadmin nebo admin pomocí formuláře vytvoří nebo upraví uživatele, zadá mu heslo a to mu předá bezpečným způsobem.

Rozhodně neplánuji žádnou self-service registraci ani reset hesla a už vůbec ne importy uživatelů. Takže opravdu velmi jednoduše. Vytvoření/editace uživatele, přiřazení/změna rolí, smazání uživatele. Ani ta deaktivace myslím není potřeba, pokud to situace bude vyžadovat, uživatele rovnou smažeme.

To je rozumné nastavení. Pro tenhle projekt bych to držel přesně tak jednoduše, jak popisuješ.

Z tvého schématu je vidět, že na to už máš dobrý základ: tabulky `users` , `roles` , `permissions` a `role_permissions` existují, a `users` už obsahuje jak webový login, tak FTP účet, hesla i `role_id` . To je pro jednoduchý adminský user management úplně dostačující.

 `press_schema`

Co bych navrhl jako Sprint 4A

Cíl:

aby `superadmin` nebo `admin` mohli v UI dělat jen tyto věci:

- zobrazit seznam uživatelů
- vytvořit uživatele
- upravit uživatele
- změnit roli
- změnit webové heslo
- změnit FTP heslo
- smazat uživatele

Bez registrace, bez resetů, bez deaktivace.

Přesný rozsah MVP

1. Seznam uživatelů

Tabulka s těmito sloupci:

- jméno a příjmení
- login (`user`)
- FTP login (`ftp_user`)
- role
- poslední přihlášení
- akce: upravit / smazat

2. Vytvoření uživatele

Formulář:

- jméno
- příjmení
- login
- webové heslo
- FTP login
- FTP heslo
- home directory
- role

3. Editace uživatele

Formulář:

- jméno
- příjmení
- login
- FTP login
- home directory
- role
- volitelně nové webové heslo
- volitelně nové FTP heslo

Když heslo necháš prázdné, nezmění se.

4. Smazání uživatele

Jednoduché potvrzení:

- opravdu smazat?
- ideálně zakázat smazání vlastního účtu
- ideálně zakázat smazání posledního `superadmin`

To bych určitě ošetřil.

Co bych neřešil teď

Souhlasím, že teď není potřeba:

- deaktivace
- self-service
- reset hesla
- import/export
- složitá správa permissions přes UI

Role už máš dané:

- `superadmin`
- `admin`
- `press_operator`
- `photographer`

a toho bych se držel.

Doporučené rozdělení práv

`superadmin`

všechno, včetně mazání uživatelů a správy rolí

`admin`

správa uživatelů, ale bez možnosti:

- smazat `superadmin`
- změnit roli `superadmin`
- povýšit někoho na `superadmin`
tohle bych nechal jen `superadminovi`

`press_operator`

bez přístupu do user managementu

`photographer`

bez přístupu do user managementu

To je jednoduché a bezpečné.

Doporučené URL / soubory

Aby to zapadlo do současné struktury, navrhoval bych:

- `users.php` — seznam uživatelů
- `user-create.php` — vytvoření
- `user-edit.php?id=...` — editace
- `user-delete.php?id=...` — smazání

A do `inc/` doplnit třeba:

- `inc/users.php` — DB funkce pro uživatele

Validace, kterou bych určitě udělal

Při vytvoření a editaci:

- `user` musí být unikátní
- `ftp_user` musí být unikátní
- `homedir` nesmí být prázdný
- `role` musí existovat
- heslo při create povinné
- heslo při editaci nepovinné
- loginy bez mezer
- smazání vlastního účtu zakázat

Dvě důležité bezpečnostní pojistky

1. Nesmí se smazat poslední `superadmin`

To je skoro povinné.

2. Admin nesmí upravit superadmina

Alespoň ne bez výslovného superadmin oprávnění.

Co bych udělal jako první implementační krok

Za mě ideální pořadí:

1. `users.php` — seznam uživatelů
2. `user-create.php`
3. `user-edit.php`
4. `user-delete.php`

Tím budeš mít během krátké chvíle funkční správu testerů.

Moje doporučení

Ano, otevřel bych teď **Sprint 4A – User management MVP** a držel ho záměrně velmi úzký a jednoduchý.

Tohle je správná chvíle, protože:

- fotoworkflow už je použitelné,
- teď potřebuješ dostat do systému lidi,
- a bez pohodlné správy uživatelů se testování zbytečně brzdí.

Pošli mi teď buď aktuální admin/menu soubory, nebo znovu balík webu, a navrhnu ti přesně první krok: `users.php` seznam uživatelů v souladu s tvou strukturou.

Zdroje

 Soubor nahrán

Jen ještě drobná oprava role photographer: bez přístupu do user managementu a bez možnosti manipulace s nahranými fotkami. Ten pouze fotky nahrává a sleduje stav (read only), co se s nimi děje.

Ano — to je velmi dobré zpřesnění a dává to smysl. 👍

Role **photographer** = **upload** + **read-only** je přesně to, co potřebuješ.

Finální model rolí

superadmin

- správa uživatelů
- správa rolí (pokud někdy bude)
- všechny fotky
- lock/unlock
- download
- admin věci

admin

- správa uživatelů (bez superadmin)
- práce s fotkami
- lock/unlock
- download
- bulk download

press_operator

- práce s fotkami
- lock/unlock
- download
- bulk download
- bez user managementu

photographer

- upload přes FTP
- vidí své fotky
- vidí stav (ready / locked / downloaded)
- **read-only**
- bez lock
- bez unlock
- bez download
- bez bulk download
- bez user managementu

To je čisté a logické.

Co to znamená v UI

Role photographer :

Na stránce photos.php :

- vidí galerii
- **nevidí klikací ready/locked**
- nevidí bulk download

- nevidí download tlačítka

Tedy:

```
ready    -> jen text  
locked   -> jen text  
downloaded -> jen text
```



bez `<a href="/select.php?...">`

A to už skoro máš — protože kód používá:

```
</> PHP
```



Stačí jen:

- `photographer` nedat permission `photos.select`
- `nedat` `photos.download`
- `nedat` `photos.bulk_download`

a UI se samo přizpůsobí.

Chování fotografa

Workflow:

Fotograf:

- nahraje přes FTP
- watcher vytvoří preview
- `status = ready`

Redaktor:

- `lockne`
- `stáhne`
- `status = downloaded`

Fotograf:

- jen sleduje, co se děje

To je přesně to, co jsi popsal.

Doporučení (malý UX bonus)

U fotografa bych navíc:

- defaultně filtroval **jen jeho fotky**
- skryl filtr fotografa
- skryl bulk toolbar

To udělá UI čistší.

Tedy:

PHP



Co dál

Ted' máme:

- workflow hotové
- role definované
- permission model jasný
- UX pro redaktory hotový

Takže opravdu je ideální pokračovat:

👉 Sprint 4A — **User management MVP**

Začneme:

1. `users.php` (seznam)
2. `user-create.php`
3. `user-edit.php`
4. `user-delete.php`

Navrhnu to tak, aby:

- superadmin viděl vše
- admin neviděl superadmin
- role se vybírala z tabulky `roles`
- heslo se zadávalo ručně

Chceš:

- jednoduchou tabulku bez CSS
- nebo rovnou ve stylu tvého admin UI?

Ano, zobrazení pro fotografa přesně jak píšeš. Vyfiltrované jen jeho fotky a co nejjednodušší zobrazení stavu, žádné další detaily. Náhledy úplně malinké, jen aby poznal o kterou fotku se jedná a celé optimalizované pro zobrazení na mobilu. Na to časem asi vytvoříme samostatnou

stránku něco jako `photos-status.php`, ale teď pojdme začít ten user management.

Přemýšlení po dobu 29s >

Souhlas. Teď je správný čas přepnout na **Sprint 4A – user management MVP**.

A беру jako závazné i to upřesnění pro `photographer` :

- bez user managementu,
- bez lock/unlock,
- bez downloadu,
- jen upload + read-only přehled stavu vlastních fotek.

Z databáze na to máš dobrý základ: `users.role_id` už odkazuje na `roles`, `users` obsahuje web login, FTP login, hash hesla i home dir, takže není potřeba nic překopávat.

 `press_schema`

Co bych teď přesně udělal

Sprint 4A scope

Jen toto:

- seznam uživatelů
- vytvoření uživatele
- editace uživatele
- smazání uživatele

Bez:

- deaktivace
- registrace
- resetu hesla
- importů

Jak to zapadne do tvé současné struktury

Podle aktuálního webu bych přidal:

- `users.php`
- `user-create.php`
- `user-edit.php`
- `user-delete.php`
- `inc/users.php`

A malou úpravu:

- `inc/header.php` — přidat odkaz **Uživatelé** jen pro `admin/superadmin`

Chování rolí

Navrhuju přesně toto:

- `superadmin` může vše
- `admin` může spravovat uživatele, ale ne `superadmin`
- `press_operator` bez user managementu
- `photographer` bez user managementu a bez manipulace s fotkami

MVP formulář uživatele

Pole:

- jméno
- příjmení
- login
- nové heslo
- FTP login
- nové FTP heslo
- home dir
- role

Při editaci:

- hesla nepovinná
- když zůstanou prázdná, nemění se

Bezpečnostní pravidla

Hned od začátku bych ošetřil:

- nelze smazat vlastní účet
- `admin` nesmí upravit ani smazat `superadmin`
- nelze smazat posledního `superadmin`

Doporučené pořadí implementace

Začal bych takto:

1. `inc/users.php` — DB funkce
2. `users.php` — seznam
3. `user-create.php`
4. `user-edit.php`
5. `user-delete.php`
6. `inc/header.php` — odkaz do menu

To je nejrychlejší cesta k tomu, abys mohl zakládat testery.

Co bude v `inc/users.php`

Připravil bych tam funkce typu:

- `users_list()`
- `users_get(int $id)`
- `roles_list()`
- `user_exists_by_login(...)`
- `user_exists_by_ftp_login(...)`
- `user_create(...)`
- `user_update(...)`
- `user_delete(...)`
- `count_superadmins()`

Moje doporučení pro úplně první krok

Začni `users.php` seznamem uživatelů. To ti okamžitě ukáže, že:

- role sedí,
- data v DB jsou v pořádku,
- UI administrace zapadá do projektu.

A hned potom přidáme create/edit.

Pošli mi teď aktuální `inc/header.php` a já ti rovnou připravím první konkrétní balík pro Sprint 4A:

- `inc/users.php`
- `users.php`
- úpravu menu v `inc/header.php`

Zdroje

 Soubor nahrán

Perfektní, díky. Menu je jednoduché, takže přidání **User managementu** bude čisté. V `header.php` máš zatím jen:

- Dashboard
- Fotografie
- Odhlásit

 header

Přidáme tam **Uživatelé**, ale pouze pro `admin` a `superadmin`.

Krok 1 — úprava header.php

Najdi blok:

```
<nav class="top-nav">
  <a href="/dashboard.php">Dashboard</a>
  <a href="/photos.php">Fotografie</a>
```

PHP



nahrad' ho tímto:

```
<nav class="top-nav">
  <a href="/dashboard.php">Dashboard</a>
  <a href="/photos.php">Fotografie</a>

  <?php if (has_permission('users.manage')): ?>
    <a href="/users.php">Uživatelé</a>
  <?php endif; ?>
```

PHP



Používáme `users.manage` jako hlavní permission pro user management.

Krok 2 — permissions pro role

Musíš dát permission:

superadmin

`users.manage`



admin

`users.manage`



press_operator

žádná

photographer

Krok 3 — vytvoříme první stránku `users.php`

Ta bude jen seznam uživatelů.

Chování:

- pouze pro `users.manage`
- tabulka
- tlačítko "Nový uživatel"
- edit
- delete

Jak bude vypadat UI

Uživatelé

+ Nový uživatel

Jméno	Login	FTP	Role	Akce
Jan Novák	novak	novak	admin	edit delete
Petr Svoboda	svoboda	svoboda	photographer	edit delete

Další krok

Ted' ti v dalším kroku pošlu:

1. `inc/users.php`
2. `users.php`
3. `user-create.php`
4. `user-edit.php`
5. `user-delete.php`

ale pošlu to postupně, ať to můžeš testovat.

Začneme:

👉 nejdřív `inc/users.php` + `users.php`

To ti hned zobrazí všechny uživatele a ověříme, že role fungují.

Výborně, header je upraven. Ještě když jsme v oblasti menu, do budoucna bude lepší, aby "homepage" byla rovnou galerie fotografií a ne dashboard, ale to je maličkost, kterou můžeme upravit kdykoliv později. Pojdme začít s těmi users.php a inc/users.php

Souhlas — homepage na photos.php je později maličkost. Teď pojdme rozjet základ user managementu. Podle tvé struktury menu a současného stylu aplikace uděláme první dvě části: inc/users.php a users.php. header.php už je jednoduchý a na to dobře naváže.

 header

Níže máš hotové celé soubory.

inc/users.php

Vytvoř nový soubor:

```
<?php
declare(strict_types=1);

require_once __DIR__ . '/db.php';

function users_can_manage(): bool
{
    return has_permission('users.manage');
}

function users_is_superadmin(array $user): bool
{
    return (($user['role_code'] ?? '') === 'superadmin');
}

function users_is_admin(array $user): bool
```

PHP

users.php

Vytvoř nový soubor:

```
<?php
declare(strict_types=1);

require_once __DIR__ . '/config/config.php';
require_once __DIR__ . '/inc/auth.php';
require_once __DIR__ . '/inc/functions.php';
require_once __DIR__ . '/inc/users.php';

require_login();

if (!has_permission('users.manage')) {
    http_response_code(403);
    exit('Přístup odepřen.');
}

$user = current_user();

</> PHP
```

Poznámky k tomu, co to dělá

Tahle první verze už řeší důležité věci:

- stránka je jen pro `users.manage`
- `superadmin` vidí všechny
- `admin` nevidí `superadmin`
- u vlastního účtu se nenabízí smazání
- `admin` nedostane možnost smazat `superadmina`

To je na první krok dobrý základ.

Volitelně: jednoduché CSS

Jestli ještě nemáš styl pro tabulku a tlačítko, přidej do `assets/style.css` :

```
.data-table {
    width: 100%;
    border-collapse: collapse;
    background: #fff;
}

.data-table th,
```

```
.data-table td {
  padding: 10px 12px;
  border-bottom: 1px solid #ddd;
  text-align: left;
  vertical-align: top;
}

.button {
```

⌕ CSS

Co dál po tomhle

Jakmile to nasadíš a ověříš, že:

- odkaz `Uživatelé` je v menu,
- seznam se správně zobrazí,
- role se filtrují správně,

navážeme dalším krokem:

- `user-create.php`
- `user-edit.php`
- `user-delete.php`

Pošli pak případně chybu nebo screenshot výsledku a navážeme přesně dál.

Zdroje

Zatím nevidím v menu položku pro user management jak si v databázi mám přidat ten permission?

Jasně — položka se v menu objeví až ve chvíli, kdy:

1. v tabulce `permissions` existuje permission `users.manage`
2. ta permission je přiřazená roli `admin` a/nebo `superadmin` v `role_permissions`

Podle tvého schématu na to už máš správné tabulky: `permissions(code, ...)`, `roles(code, ...)` a vazební `role_permissions(role_id, permission_id, allowed)`.

📄 `press_schema`

Nejjednodušší postup

1. Přidej permission

```
INSERT INTO permissions (code, name, description)
VALUES (
    'users.manage',
    'Správa uživatelů',
```

SQL



2. Přřad' ji roli superadmin

```
INSERT INTO role_permissions (role_id, permission_id, allowed)
SELECT r.id, p.id, 1
FROM roles r
```

SQL



3. Přřad' ji roli admin

```
INSERT INTO role_permissions (role_id, permission_id, allowed)
SELECT r.id, p.id, 1
FROM roles r
```

SQL



Bezpečnější varianta proti duplicitám

Kdybys to spouštěl opakovaně, je lepší použít:

```
INSERT IGNORE INTO permissions (code, name, description)
VALUES (
    'users.manage',
    'Správa uživatelů',
```

SQL



a pak:

```
INSERT IGNORE INTO role_permissions (role_id, permission_id, allowed)
SELECT r.id, p.id, 1
```



```
FROM roles r
```

```
</> SQL
```



To je asi nejlepší varianta.

Jak si ověříš, že je to zapsané

```
SELECT
    r.code AS role_code,
    p.code AS permission_code,
    rp.allowed
FROM role_permissions rp
JOIN roles r ON r.id = rp.role_id
```

```
</> SQL
```



Měl bys dostat aspoň:

- superadmin / users.manage / 1
- admin / users.manage / 1

Když se to pořád neukáže v menu

Pak zkontroluj ještě dvě věci:

1. Je v `header.php` opravdu tato podmínka?

```
<?php if (has_permission('users.manage')): ?>
```

```
</> PHP
```



2. Odhlásit a znovu přihlásit

Pokud se permissions cachují do session, může být potřeba nový login.

Doporučený jednorázový blok

Tohle můžeš pustit celé najednou:

```
INSERT IGNORE INTO permissions (code, name, description)
VALUES (
    'users.manage',
    'Správa uživatelů',
    'Vytváření, úprava a mazání uživatelů'
);

INSERT IGNORE INTO role_permissions (role_id, permission_id, allowed)
SELECT r.id, p.id, 1
FROM roles r
```

SQL



Po tom se odhlaš a znovu přihlas.

Pak by se měla položka **Uživatelé** objevit.

Zdroje

Výborně, musel jsem se odhlásit a přihlásit, ale položku uživatelé už vidím.

Tabulka uživatelů je zatím velmi nepřehledná a odkazy jsou tmavě modrým písmem na černém pozadí, ale to předpokládám teprve budeme řešit.

Přesně tak — tohle je teď normální stav. Funkčnost jsme právě ověřili, teď přijde na řadu **doladění UI a ergonomie**.

To, co popisuješ, jsou v zásadě dvě oddělené věci:

- vizuální styl tabulky a odkazů
- obsahová přehlednost tabulky

A já bych je řešil v tomto pořadí:

1. Nejdřív zlepšit přehlednost tabulky

Tj. upravit, co se vlastně zobrazuje.

U `users.php` bych doporučil zjednodušit seznam na něco jako:

- Jméno
- Login
- Role

- FTP login
- Poslední přihlášení
- Akce

A naopak bych z hlavního seznamu klidně vyhodil:

- `homedir` z hlavní tabulky
- případně další technické údaje

`homedir` je důležitý při editaci, ale v seznamu spíš překáží.

Také bych:

- sjednotil šířky sloupců,
- akce dal do jednoho kompaktního sloupce,
- zvýraznil roli.

2. Pak doladit CSS

To tmavě modré na černém pozadí je klasický důsledek toho, že se používá default styl odkazu v tmavém layoutu.

To se spraví snadno:

- tabulka dostane vlastní styl,
- odkazy v tabulce dostanou světlou barvu,
- akční odkazy budou vypadat jako malé buttony nebo badge.

Co bych navrhl jako další konkrétní krok

Ted' bych udělal:

Sprint 4A.1 – polish `users.php`

- zpřehlednit sloupce
- zlepšit akce
- doplnit základní CSS pro tabulku
- opravit barvy odkazů

A hned potom:

Sprint 4A.2 – `user-create.php`

- formulář pro založení uživatele

To je ideální tempo:

nejdřív udělat hezký a použitelný seznam, pak na něj navázat create/edit.

Co od tebe teď potřebuji

Pošli mi prosím aktuální soubor:

- `assets/style.css`
- a klidně i aktuální `users.php` , pokud sis ho už nějak upravoval

Pak ti připravím:

- upravené `users.php`
- konkrétní CSS blok pro tabulku uživatelů
- a hned navážeme `user-create.php`

 Soubor nahrán

Možná bych z tabulky vypustil FTP login, ten může být vidět až při rozkliknutí uživatele v detailech. U většiny uživatelů bude shodný s loginem na web. `users` jsem zatím neupravoval a `style.css` posílám.

Přemýšlel >

Přesně tak — problém s tmavě modrými odkazy na černém pozadí dělá to, že `data-table` má teď v CSS bílé pozadí a odkazy se pak barví browserovým defaultem. Je to vidět přímo v `style.css` , kde je:

- `.data-table { background: #fff; }`
- `.button { background: #f7f7f7; color: inherit; }`

To v tmavém layoutu neseď.  `style`

A souhlasím i s tím, že **FTP login z hlavní tabulky pryč**. V seznamu uživatelů je zbytečný.

Co bych teď udělal

1. upravit `users.php` , aby bylo přehlednější
2. přepsat CSS pro tabulku do tmavého stylu

Nové `users.php`

Nahrad' celý soubor tímto:

```
<?php
declare(strict_types=1);

require_once __DIR__ . '/config/config.php';
require_once __DIR__ . '/inc/auth.php';
```



```
require_once __DIR__ . '/inc/functions.php';
require_once __DIR__ . '/inc/users.php';

require_login();

if (!has_permission('users.manage')) {
    http_response_code(403);
    exit('Přístup odepřen.');
}
```

</> PHP



Co přidat do style.css

Na konec assets/style.css přidej tento blok:

```
.page-head {
    display: flex;
    justify-content: space-between;
    align-items: center;
    gap: 16px;
    margin-bottom: 18px;
}

.table-wrap {
    overflow-x: auto;
}

.data-table {
    width: 100%;
    border-collapse: collapse;
    background: var(--bg-card);
}
```

</> CSS



Co je dobré zároveň odstranit

Ve style.css teď máš starý blok:

```
.data-table {
    width: 100%;
    border-collapse: collapse;
    background: #fff;
}
```

```
}

.data-table th,
.data-table td {
    padding: 10px 12px;
    border-bottom: 1px solid #ddd;
    text-align: left;
    vertical-align: top;
}

.button {
```

</> CSS

Ten buď smaž, nebo nech přepsat novým blokem na konci souboru. Teď je to právě ten kus, který rozbíjí tmavý vzhled.  style

Výsledek

Po této úpravě bude seznam:

- přehlednější
- bez FTP loginu
- barevně sladěný s tmavým layoutem
- použitelný i pro další kroky

Pak bych hned pokračoval `user-create.php`.

Zdroje

Krása! teď je tabulka perfektní. A je výborný nápad to barevně odlišením rolí!

Super — to jsem rád, že sedí. Barevné role se pak hodí i na dalších místech (detail uživatele, audit log, případně i u fotek „zamčeno kým“).

Teď plynule navážeme na **Sprint 4A.2 — vytvoření uživatele**.

Další krok bude `user-create.php`, aby šlo hned zakládat testery.

Co bude umět `user-create.php`

Formulář:

- jméno
- příjmení

- login
- heslo
- role (select z tabulky roles)
- FTP login (default = login)
- FTP heslo
- home dir (default např. /data/ftp/{ftp_user})

Validace:

- login unikátní
- ftp_login unikátní
- heslo povinné
- role musí existovat
- admin nesmí vytvořit superadmin

Chování:

- po uložení → redirect na `users.php`
- zobrazí se nový uživatel v tabulce

Malý UX detail (doporučuji)

Při zadání loginu:

automaticky vyplnit:

- FTP login = login
- home dir = /data/ftp/login

ale umožnit ruční změnu.

To zrychlí zakládání uživatelů.

Hashování hesel

Použijeme:

```
</> PHP
```



a totéž pro FTP heslo.

DB insert

Bude vypadat přibližně:

```
INSERT INTO users
```

```
</> SQL
```



Pořadí implementace

Ted' navrhuju:

1. `user-create.php`
2. přidat tlačítko „Zpět na seznam“
3. validace chyb (červené boxy)
4. redirect po uložení

Po tom hned:

- `user-edit.php`
- `user-delete.php`

Chceš:

- create + edit ve stejném formuláři
nebo
- oddělené soubory (jednodušší, čitelnější)?

Doporučuji **oddělené**, je to přehlednější a méně magie.

Klidně oddělené soubory. Můžeme jít na implementaci. Jen upozorňuji, že hesla doposud ručně vytvořených uživatelů v databázi byla vytvářena pomocí funkce `password_hash('silneheslo', PASSWORD_BCRYPT)` nikoliv tebou navrhované `PASSWORD_DEFAULT`.

Přemýšlení po dobu 18s >

Jasně — díky za upřesnění. Budeme se držet `PASSWORD_BCRYPT`, aby to bylo konzistentní s tím, co už v databázi máš.

Z dumpu je vidět, že `users` tabulka přesně odpovídá tomu, co potřebujeme pro create formulář: `jmeno`, `prijmeni`, `user`, `pass_hash`, `ftp_user`, `ftp_pass_hash`, `homedir`, `role_id` a vazba na `roles`.  `press_schema`

Půjdeme rovnou na implementaci.

1. Nové celé znění `inc/users.php`

Nahrad' celý soubor tímto:


```
<?php
declare(strict_types=1);

require_once __DIR__ . '/db.php';

function users_can_manage(): bool
{
    return has_permission('users.manage');
}

function users_is_superadmin(array $user): bool
{
    return (($user['role_code'] ?? '') === 'superadmin');
}

function users_is_admin(array $user): bool

</> PHP
```

2. Nový soubor user-create.php

Vytvoř nový soubor:

```
<?php
declare(strict_types=1);

require_once __DIR__ . '/config/config.php';
require_once __DIR__ . '/inc/auth.php';
require_once __DIR__ . '/inc/functions.php';
require_once __DIR__ . '/inc/users.php';

require_login();

if (!has_permission('users.manage')) {
    http_response_code(403);
    exit('Přístup odepřen.');
```

```
$roles = users_roles_list();
```

```
</> PHP
```

3. Přidej do style.css

Na konec `assets/style.css` přidej:

```
.user-form label {
  display: block;
  margin-bottom: 6px;
  color: var(--text-muted);
  font-size: 14px;
  font-weight: 600;
}

.user-form input,
.user-form select {
  width: 100%;
  padding: 10px 12px;
  border: 1px solid var(--border);
  border-radius: 6px;
  font-size: 15px;
  background: #111;
}
```

⌄ CSS



Co to bude umět

Tahle verze:

- vytvoří nového uživatele
- použije `PASSWORD_BCRYPT` pro web i FTP heslo
- dovolí adminovi jen role bez `superadmin`
- defaultně doplní:
 - `ftp_user = login`
 - `homedir = /home/ftp/login`
- po uložení vrátí na seznam uživatelů

Drobnost k home directory

Ted' jsem nastavil default:

`/home/ftp/{login}`



Jestli chceš jinou konvenci, třeba:

`/var/presscentrum/incoming/{login}`



nebo cokoli, upravíme jen tento řádek v JS:

JavaScript



Až to nasadíš a vyzkoušíš, půjdeme na `user-edit.php`.

Zdroje

Správná cesta k FTP je `/var/www/press/ftp/login` Kde je ten javascript, ve kterém to mám opravit?

Ten JavaScript je **na konci souboru** `user-create.php`, úplně dole pod formulářem, těsně před:

PHP



Najdeš tam blok:

```
<script>
document.addEventListener('DOMContentLoaded', function () {
    const loginInput = document.getElementById('user');
    const ftpLoginInput = document.getElementById('ftp_user');
    const homedirInput = document.getElementById('homedir');

    function slugify(value) {
        return value.trim().replace(/\\s+/g, '');
    }

    function syncDefaults() {
        const login = slugify(loginInput.value);

        if (ftpLoginInput.value.trim() === '' || ftpLoginInput.dataset.autofill:
            ftpLoginInput.value = login;
            ftpLoginInput.dataset.autofill = '1':
    }
});
```

HTML



A tenhle řádek změň:

JavaScript



na:

JavaScript



Výsledný správný blok

```
if (homedirInput.value.trim() === '' || homedirInput.dataset.autofill === '1') {  
    homedirInput.value = login ? '/var/www/press/ftp/' + login : '';  
    homedirInput.dataset.autofill = '1';  
}
```

JavaScript



Co to pak dělá

Když zadáš login:

novak



automaticky se doplní:

FTP login:

novak



Home dir:

/var/www/press/ftp/novak



Ale jakmile to ručně změníš, autofill už se nepřepisuje.

Výborně, opraveno a user-create funguje. Jen je potřeba opravit zadávání hesel, aby ve formuláři bylo pole typu password a bylo vidět jen psaní puntíků a nikoliv text

hesla. Ideální by bylo, kdyby tam k tomu byla ikonka oka k dočasnému zobrazení pro kontrolu. A také u hesla by bylo dobré kopírovat položku password do položky ftppassword s možností případné úpravy.

Skvělé. To jsou přesně správné UX úpravy.

Uděláme tři věci:

- obě hesla budou `type="password"`
- přidáme tlačítko s okem pro zobrazení/skrytí
- FTP heslo se bude automaticky kopírovat z webového hesla, dokud ho ručně nezměníš

Co upravit v `user-create.php`

1. Změň pole hesel

Najdi tento blok:

```
<div>
    <label for="password">Webové heslo</label>
    <input type="text" name="password" id="password" value="<?=$_h($values['password'])" />
</div>

<div>
    <label for="ftp_password">FTP heslo</label>
    <input type="text" name="ftp_password" id="ftp_password" value="<?=$_h($values['ftp_password'])" />
</div>
```

PHP

Nahraď ho tímto:

```
<div>
    <label for="password">Webové heslo</label>
    <div class="password-wrap">
        <input type="password" name="password" id="password" value="<?=$_h($values['password'])" />
        <button type="button" class="password-toggle" data-target="password" aria-label="Toggle password visibility" />
    </div>
</div>

<div>
    <label for="ftp_password">FTP heslo</label>
    <div class="password-wrap">
        <input type="password" name="ftp_password" id="ftp_password" value="<?=$_h($values['ftp_password'])" />
        <button type="button" class="password-toggle" data-target="ftp_password" aria-label="Toggle ftp password visibility" />
    </div>
</div>
```

```
<button type="button" class="password-toggle" data-target="ftp_password"
...>
```

</> PHP



2. Uprav JS na konci user-create.php

Na konci souboru už máš `<script>...</script>`. Nahraď celý ten blok tímto:

```
<script>
document.addEventListener('DOMContentLoaded', function () {
  const loginInput = document.getElementById('user');
  const passwordInput = document.getElementById('password');
  const ftpLoginInput = document.getElementById('ftp_user');
  const ftpPasswordInput = document.getElementById('ftp_password');
  const homedirInput = document.getElementById('homedir');

  function slugify(value) {
    return value.trim().replace(/\\s+/g, '');
  }

  function syncDefaults() {
    const login = slugify(loginInput.value);

    if (ftpLoginInput.value.trim() === '' || ftpLoginInput.dataset.autofill)

```

</> HTML



Co přidat do style.css

Na konec `assets/style.css` přidej:

```
.password-wrap {
  display: flex;
  align-items: stretch;
  gap: 8px;
}

.password-wrap input {
  flex: 1 1 auto;
}

.password-toggle {
  flex: 0 0 auto;
  min-width: 44px;
}
```

```
border: 1px solid var(--border);
border-radius: 6px;
background: #111;
```

CSS



Jak se to bude chovat

Když začneš psát do webové heslo :

- FTP heslo se automaticky vyplní stejnou hodnotou

Pokud pak FTP heslo ručně změníš:

- přestane se automaticky přepisovat

A obě pole půjdou tlačítkem oka dočasně zobrazit.

Poznámka k ikoně oka

Použil jsem emoji  a  , protože je to rychlé a bez dalších ikonových knihoven.

Později to můžeme udělat hezčí přes SVG nebo vlastní ikonu v CSS.

Další logický krok je `user-edit.php` , kde uděláme stejný UX model pro změnu hesel.

Výborně, funguje to, ikonky zcela vyhovují. udělejme `user-edit.php`

Přemýšlení po dobu 29s >

Skvělé. Uděláme `user-edit.php` tak, aby:

- načetl existujícího uživatele,
- umožnil změnit běžné údaje,
- hesla měnil jen když se vyplní,
- používal `PASSWORD_BCRYPT` ,
- admin nemohl upravovat `superadmin` ,
- nešlo odrolovat posledního `superadmina` .

Bude potřeba:

1. doplnit funkci `users_update()` do `inc/users.php`
2. vytvořit nový soubor `user-edit.php`

1. Doplně do `inc/users.php` funkci `users_update()`

Na konec souboru `inc/users.php` přidej tento blok:

```
function users_update(int $id, array $data): void
{
    $pdo = db();

    $fields = [
        'jmeno = :jmeno',
        'prijmeni = :prijmeni',
        'user = :user',
        'ftp_user = :ftp_user',
        'homedir = :homedir',
        'role_id = :role_id',
    ];

    $params = [
        ':id' => $id,
        ':jmeno' => $data['jmeno'],
    ];
}
```

PHP



2. Vytvoř nový soubor `user-edit.php`

```
<?php
declare(strict_types=1);

require_once __DIR__ . '/config/config.php';
require_once __DIR__ . '/inc/auth.php';
require_once __DIR__ . '/inc/functions.php';
require_once __DIR__ . '/inc/users.php';

require_login();

if (!has_permission('users.manage')) {
    http_response_code(403);
    exit('Přístup odepřen.');
}

$currentUser = current_user();
```

PHP



3. Přidej do `style.css` ještě tento drobný blok

Jestli tam ještě není, přidej na konec:

```
.form-help {  
  display: block;  
  margin-top: 6px;  
  color: var(--text-muted);
```

⌞ CSS



Co teď umí `user-edit.php`

- upravit jméno, příjmení, login, FTP login, home dir, roli
- změnit web/FTP heslo jen pokud se něco vyplní
- při zadání nového webového hesla automaticky kopírovat do FTP hesla
- přitom ponechat možnost FTP heslo ručně změnit
- chránit posledního `superadmina`

Doporučený další krok

Po otestování bych hned navázal `user-delete.php`, aby byl základ user managementu kompletní.

Funguje to. Jen nějakou změnou ve stylech teď došlo k tomu, že políčka pro zadávání hesel jsou vyšší než ostatní a také se zvětšila ikonka oka. Předtím byly řádky stejně vysoké a ikonka menší a vypadalo to lépe. Až to opravíme, můžeme se pustit do `user-delete.php`

To bude přesně ono: tlačítko oka ti nejspíš začalo dědit styl z tohoto globálního pravidla:

```
.form button {  
  margin-top: 10px;  
  padding: 12px;  
  border: 0;  
  border-radius: 6px;  
  background: var(--accent);
```

```
color: #000;  
font-weight: bold;
```

CSS



Proto je oko větší a celý řádek vyšší.

Oprava ve `style.css`

Na konec `assets/style.css` přidej tento blok:

```
.password-wrap {  
  display: flex;  
  align-items: stretch;  
  gap: 8px;  
}  
  
.password-wrap input {  
  flex: 1 1 auto;  
  margin: 0;  
}  
  
.password-wrap .password-toggle {  
  flex: 0 0 40px;  
  width: 40px;  
  min-width: 40px;  
  margin-top: 0;
```

CSS



Ještě doporučuji odstranit starší blok

Jestli ve `style.css` už máš dříve přidaný blok:

```
.password-wrap {  
  display: flex;  
  align-items: stretch;  
  gap: 8px;  
}  
  
.password-wrap input {  
  flex: 1 1 auto;  
}  
  
.password-toggle {  
  flex: 0 0 auto;  
  min-width: 44px;  
  border: 1px solid var(--border);  
  border-radius: 6px;
```

tak ho smaž a nech jen tu novou verzi výše.

Proč to funguje

Tímhle:

- zrušíš `margin-top` zděděný z `.form button`
- zrušíš velké `paddingy`
- zmenšíš ikonku oka
- výška řádku se srovná s ostatními inputy

Až to doladíš, půjdeme na `user-delete.php`.

Ano, teď je to perfektní. Pojdme na `user-delete`

Skvělé. Uděláme `user-delete.php` jednoduše a bezpečně:

- smaže uživatele až po potvrzení
- nepůjde smazat vlastní účet
- `admin` nesmí smazat `superadmin`
- nepůjde smazat posledního `superadmina`

Budou potřeba 2 kroky:

1. doplnit `users_delete()` do `inc/users.php`
2. vytvořit `user-delete.php`

1. Dopln do `inc/users.php` funkci `users_delete()`

Na konec souboru `inc/users.php` přidej:

```
function users_delete(int $id): void
{
    $pdo = db();

    $stmt = $pdo->prepare("
        DELETE FROM users
        WHERE id = ?
        LIMIT 1
    ");
```

```
}
```

2. Vytvoř nový soubor user-delete.php

```
<?php
declare(strict_types=1);

require_once __DIR__ . '/config/config.php';
require_once __DIR__ . '/inc/auth.php';
require_once __DIR__ . '/inc/functions.php';
require_once __DIR__ . '/inc/users.php';

require_login();

if (!has_permission('users.manage')) {
    http_response_code(403);
    exit('Přístup odepřen.');
}

$currentUser = current_user();

</> PHP
```

3. Přidej do style.css styl pro mazací tlačítko

Na konec assets/style.css přidej:

```
.btn-danger {
    background: #7f1d1d;
    color: #fecaca;
    border: 1px solid #b91c1c;
}

.btn-danger:hover {
    background: #991b1b;
}

</> CSS
```

Jak to bude fungovat

Ze seznamu uživatelů klikneš na `Smazat` :

- otevře se potvrzovací stránka
- zobrazí se základní údaje uživatele
- tlačítko:
 - Ano, smazat uživatele
 - Zrušit

A bezpečnostní omezení zůstanou zachována.

Doporučení po nasazení

Vyzkoušej tyto 4 scénáře:

1. smazání běžného uživatele adminem
2. pokus o smazání vlastního účtu
3. pokus admina smazat superadmina
4. pokus smazat posledního superadmina

Když to projde, bude základ user managementu hotový.

Další logický krok pak bude drobný polish:

- flash hlášky po create/edit/delete
- případně detail uživatele, pokud ho budeš chtít.

Funguje to, ale je tam jedna nepěkná drobnost. Při kliknutí na `Smazat` se nejprve zobrazí bílý dialog (evidentně bez stylů) `Web press.clovekavira.cz` říká

Opravdu smazat uživatele user? (OK) (Zrušit)
a teprve po odkliknutí OK to skočí na skutečný formulář smazání uživatele s červeným tlačítkem (Ano, smazat uživatele).

Můžeme se zbavit toho otravného mezikroku?

Ano — ten bílý dialog je z `onclick="return confirm(...)"` přímo v odkazu na stránce `users.php` . Takže teď máš **dvojí potvrzení**:

1. browserový `confirm()`
2. skutečnou stránku `user-delete.php`

To první už je zbytečné. Stačí nechat jen druhé, hezčí potvrzení.

Co změnit v `users.php`

Najdi v odkazu na smazání tento blok:

```
<a class="table-action table-action-danger"
  href="/user-delete.php?id=<?= (int)$u['id'] ?>"
  onclick="return confirm('Opravdu smazat uživatele <?= h((string)$u['user']) ?>') ?>"
  Smazat
```

PHP



A změň ho na:

```
<a class="table-action table-action-danger"
  href="/user-delete.php?id=<?= (int)$u['id'] ?>">
```

PHP



To je celé.

Proč to funguje

Pak bude flow už jen:

- klik na Smazat
- otevře se user-delete.php
- tam uvidíš stylovanou potvrzovací stránku
- Ano, smazat uživatele / Zrušit

Bez otravného mezikroku.

Drobné doporučení navíc

Stejný princip bych držel i dál:

- u destruktivních akcí v tabulkách nepoužívat browserový `confirm()`
- raději vždy vlastní potvrzovací stránku nebo modal

Je to čistší a konzistentnější.

Až to opraviš, máš tím v zásadě hotový základ Sprintu 4A. Další logický krok bude drobný polish: flash hlášky po create/edit/delete a pak případně stránka pro fotografa `photos-status.php`.

User management funguje bezvadně. vyzkoušel jsem všechny role uživatelů s křížovým prozkoušením práv v user managementu i při správě galerie. Vše je z hlediska

práv naprosto v pořádku a každý vidí jen to, co má vidět. Mám jedinou drobnou připomínku. Když redaktor zamkne některé fotky, vidí u nich příznak "ke stažení". To je naprosto správně. Když se ale přihlásí jiný uživatel s rolí umožňující zamykání fotek, vidí tam také nápis "ke stažení". Bylo by lepší, kdyby cizí redaktor viděl místo toho nápis "zamknuto" a jen u těch, které si označí sám by viděl "ke stažení". Je to jen otázka zobrazení toho labelu, práva fungují dobře s cizí zamčenou fotkou jiný redaktor manipulovat nemůže. Poradíš, kde to opravit?

Ano — tohle je jen drobná úprava zobrazení v `photos.php`.

Podle aktuální verze se label pro zamčené fotky rozhoduje podle `locked_by_user_id`, ale text/klasa pro cizí zamčené a moje zamčené je potřeba odlišit. V nahraném `photos.php` je to právě v bloku uvnitř `.status-wrapper`, kde se řeší `downloaded`, pak

`locked_by_user_id`, a pak větev pro aktuálního uživatele vs. ostatní.  `photos`

Kde to opravit

V `photos.php` najdi tento blok:

```
<?php elseif ($p['locked_by_user_id']): ?>

<?php if ($p['locked_by_user_id'] == $currentUserId): ?>

<a href="/select.php?id=<?=(int)$p['id'] ?>&action=unlock"
class="status status-selected status-clickable"
onclick="event.stopPropagation();">
locked
</a>

<?php else: ?>

<div class="status status-locked">
locked
</div>
```

 PHP



Co tam má být

Uprav ho tak, aby:

- moje zamčené fotky zobrazovaly **ke stažení**
- cizí zamčené fotky zobrazovaly **zamknuto**

Tedy takto:

```
<?php elseif ($p['locked_by_user_id']): ?>

<?php if ($p['locked_by_user_id'] == $currentUserId): ?>

<a href="/select.php?id=<?=(int)$p['id'] ?>&action=unlock"
class="status status-selected status-clickable"
onclick="event.stopPropagation();">
ke stažení
</a>

<?php else: ?>

<div class="status status-locked">
zamknuto
</div>

</> PHP
```



Co to udělá

Pak bude chování přesně toto:

- **můj lock** → ke stažení
- **cizí lock** → zamknuto
- **ready** → ready
- **downloaded** → downloaded

Volitelně ještě hezčí varianta

Kdybys chtěl, můžeš později přepsat i ostatní texty do češtiny:

- **ready** → připraveno
- **downloaded** → staženo

Ale to už je čistě kosmetika.

Tohle je jediná změna, kterou teď potřebuješ.

Zdroje

Super, zafungovalo. Ještě drobnost k té stejné situaci. Fotky ve stavu "zamknuto" mají stejný žlutý rámeček jako fotky ve stavu "ke stažení". Můžeme upravit styly tak, aby fotky ve stavu "zamknuto" měly sice žlutý rámeček, ale ve

značně dimmed stavu oproti výraznému žlutému rámečku
fotek "ke stažení"?

Ano — tohle je čistě CSS úprava a je to dobrý nápad 👍

Rozlišíme:

- **ke stažení (moje locked)** → výrazný žlutý rámeček
- **zamknuto (cizí locked)** → tlumený / dimmed žlutý rámeček

Stačí upravit `style.css`.

Najdi v `style.css` tyto styly

Pravděpodobně máš něco jako:

```
.photo-card.selected {  
  border-color: var(--accent);  
}
```

```
.status-selected {
```

⌄ CSS



a zároveň:

```
.status-locked {
```

⌄ CSS



Proto vypadají stejně.

Přidej dimmed variantu pro "zamknuto"

Na konec `style.css` přidej:

```
.photo-card.locked {  
  border-color: rgba(234, 179, 8, 0.35);  
  box-shadow: none;  
  opacity: 0.85;  
}
```

```
.status-locked {  
  color: rgba(234, 179, 8, 0.65);
```

⌄ CSS



```
}
```

A zkontroluj že `photos.php` používá správné třídy

Moje locked (ke stažení):

```
</> PHP
```



Cizí locked:

```
</> PHP
```



Pokud tam máš zatím oboje `selected`, uprav to v `photos.php`:

Najdi:

```
if ($p['locked_by_user_id'] == $currentUserId) {  
    $cardClass .= ' selected';  
} else {
```

```
</> PHP
```



a změň na:

```
if ($p['locked_by_user_id'] == $currentUserId) {  
    $cardClass .= ' selected';  
} else {
```

```
</> PHP
```



Výsledek

Stav	Rámeček
ready	normální
ke stažení	výrazný žlutý
zamknuto	tlumený žlutý

downloaded

zelený (už máš)

UX pak bude velmi čitelný:

- moje → svítí
- cizí → jen naznačeno
- ready → neutrální

 Soubor nahrán

tady tě poprosím, abys to upravil sám moc se tam v tom souboru nevyznám. posílám soubor. prosím pošli mi celkovou správnou verzi.

Přemýšlel >

Jasně — tady je **celé správné nové znění** `photos.php`. Udělal jsem v něm dvě věci:

- **moje zamčené** fotky mají třídu `selected`
- **cizí zamčené** fotky mají třídu `locked`

Tím půjde v CSS odlišit výrazný a tlumený žlutý rámeček.

Zároveň jsem opravil i drobnou chybu v JS, kde chyběla proměnná `current`. V nahraném souboru byla použita, ale nebyla definovaná.  `photos`

Nové celé znění `photos.php`

```
<?php
declare(strict_types=1);

require_once __DIR__ . '/config/config.php';
require_once __DIR__ . '/inc/auth.php';
require_once __DIR__ . '/inc/functions.php';
require_once __DIR__ . '/inc/photos.php';

require_login();

if (!has_permission('photos.view')) {
    http_response_code(403);
    exit('Přístup odepřen.');
```

A ještě potřebný CSS doplněk

Aby se to opravdu vizuálně projevilo, přidej do `style.css` tento blok:

```
.photo-card.selected {  
  border-color: #f59e0b;  
  box-shadow: 0 0 0 1px #f59e0b;  
}  
  
.photo-card.locked {  
  border-color: rgba(245, 158, 11, 0.35);  
  box-shadow: 0 0 0 1px rgba(245, 158, 11, 0.18);  
}  
  
.status-locked {  
  background: #111;  
  color: rgba(245, 158, 11, 0.70);  
}
```

</> CSS



Pokud už ve `style.css` máš starší definice `.photo-card.selected` nebo `.status-locked`, nech tam jen jednu finální verzi, aby se navzájem nepřebíjely.

Až tohle nasadiš, budou:

- **ke stažení** = výrazně žluté
- **zamknuto** = tlumeně žluté

a bude to vizuálně mnohem čitelnější.

[Zdroje](#)

Výborně, styly odladěny. Poslední informace, která by se moc hodila je, aby redaktor viděl, kým byla cizí zamknutá fotka zamknuta. Bylo by možné vedle tlačítka zamknuto zobrazit třeba "(Jméno Příjmení)" redaktora, který fotku zamknul?

Ano — a dobrá zpráva je, že to půjde udělat jen malou úpravou `photos.php`. Není potřeba měnit databázi.

Co změnit v `photos.php`

Najdi tento blok:

```
<?php else: ?>

<div class="status status-locked">
    zamknuto
</div>
```

PHP



A nahrad' ho tímto:

```
<?php else: ?>

<div class="status-line">
    <div class="status status-locked">
        zamknuto
    </div>

    <?php
    $lockedByName = trim(
        ((string)($p['locked_jmeno'] ?? '')) . ' ' .
        ((string)($p['locked_prijmeni'] ?? ''))
    );
    ?>

    <?php if ($lockedByName !== ''): ?>
        <div class="lock-owner">
```

PHP



Tím dostaneš třeba:

zamknuto (Jan Novák)



a když by jméno chybělo, spadne to aspoň na login.

Co přidat do `style.css`

Na konec `assets/style.css` přidej:

```
.status-line {  
  display: flex;  
  align-items: center;  
  gap: 6px;  
  flex-wrap: wrap;  
}
```

```
.lock-owner {  
  font-size: 12px;
```

⌄ CSS



Jak to bude fungovat

- moje zamčené: pořád jen ke stažení
- cizí zamčené: zamknuto (Jméno Příjmení)
- když jméno nebude dostupné: zamknuto (login)

Když budeš chtít, další drobný krok může být, že to jméno bude ještě jemně barevně odlišené podle role, ale teď bych to nechal takhle čisté.

Úplně perfektní, teď ještě prosím stejnou úpravu v
photo.php

Ano, uděláme to stejně i v detailu fotografie (photo.php).

Abych to upravil přesně (tam bývá jiná struktura HTML), pošli mi prosím aktuální soubor **photo.php** — stejně jako jsi poslal photos.php .

Jakmile ho pošleš:

- doplním zamknuto (Jméno Příjmení)
- zachovám stejné CSS (status-line , lock-owner)
- sjednotím styl s galerií
- pošlu ti celé hotové znění souboru

📁 Soubor nahrán

Jasně — tady je **celé správné nové znění** photo.php se stejnou úpravou jako v galerii:

- moje zamčená fotka → ke stažení
- cizí zamčená fotka → zamknuto (Jméno Příjmení)
- když jméno nebude k dispozici, spadne to na login

Vycházím z tebou nahraného aktuálního souboru.  photo

```
<?php
declare(strict_types=1);

require_once __DIR__ . '/config/config.php';
require_once __DIR__ . '/inc/auth.php';
require_once __DIR__ . '/inc/functions.php';
require_once __DIR__ . '/inc/photos.php';

require_login();

if (!has_permission('photos.view')) {
    http_response_code(403);
    exit('Přístup odepřen.');
```

PHP

Tohle bude fungovat se stejnými CSS třídami, které už máš přidané pro galerii:

CSS

Takže ve style.css už nic dalšího přidávat nemusíš, pokud jsi ty třídy přidal při předchozí úpravě.

Zdroje

Ne tak tohle nefunguje. Neposlal jsi mi náhodou zpět tu
mojí původní verzi photo.php?

Máš pravdu — neposlal jsem ti zpět původní soubor, ale poslal jsem **neúplnou úpravu**. V `photo.php` jsem použil pole `locked_jmeno`, `locked_prijmeni` a `locked_by_user`, ale `photos_get_by_id()` je zatím vůbec nenačítá. Proto to nemohlo fungovat. V `photo.php` jsou ta pole opravdu používána jen v mé navržené úpravě, ale původní `photos_get_by_id()` vrací jen `p.*` a autora přes `p.user_id`.  photo

Takže je potřeba upravit **2 soubory**:

1. `inc/photos.php`
2. `photo.php`

Níže máš přesně celé správné znění obou částí.

1. V `inc/photos.php` nahrad' funkci `photos_get_by_id()` touto verzí

Najdi stávající funkci:

```
function photos_get_by_id(int $id): ?array
{
```

</> PHP



a nahrad' ji celou tímto:

```
function photos_get_by_id(int $id): ?array
{
    $sql = "
        SELECT
            p.*,
            u.jmeno,
            u.prijmeni,
            u.user AS web_user,
            lu.user AS locked_by_user,
            lu.jmeno AS locked_jmeno,
            lu.prijmeni AS locked_prijmeni
        FROM photos p
        LEFT JOIN users u ON u.id = p.user_id
        LEFT JOIN users lu ON lu.id = p.locked_by_user_id
        WHERE p.id = :id
    "
    $stmt = $db->prepare($sql);
    $stmt->execute(['id' => $id]);
    $result = $stmt->fetch_all();
    return $result;
}
```

</> PHP



2. Kompletní správné znění photo.php

Nahradí celý soubor tímto:

```
<?php
declare(strict_types=1);

require_once __DIR__ . '/config/config.php';
require_once __DIR__ . '/inc/auth.php';
require_once __DIR__ . '/inc/functions.php';
require_once __DIR__ . '/inc/photos.php';

require_login();

if (!has_permission('photos.view')) {
    http_response_code(403);
    exit('Přístup odepřen.');
}

$id = (int)($_GET['id'] ?? 0);

</> PHP
```

Proč to teď bude fungovat

Teď se to sjednotí s galerií:

- photos.php už jméno lockujícího uživatele má
- photo.php ho teď nově dostane také, protože photos_get_by_id() přidá join na users lu
- detail pak může korektně vykreslit:
 - ke stažení
 - nebo zamknuto (Jméno Příjmení)

Co zkontrolovat po nasazení

Otevři detail fotky, kterou:

- zamknul jiný redaktor
a měl bys vidět:

zamknuto (Jan Novák)

Když jméno není vyplněné, ukáže se aspoň login v závorce.

Tohle je ta správná oprava.

Zdroje

Perfektní, funguje. Tím máme v podstatě hlavní funkcionální systém implementovanou. Pojdme ještě v této chvíli vyřešit to, aby se po přihlášení uživatelům rovnou zobrazila galerie a ne dashboard. Uživatel s rolí fotograf by se měla namísto plnohodnotné galerie zobrazit redukováná "read-only" verze fotogalerie laděná na mobily, ale tuhle stránku zatím ještě nemáme, takže to změníme až později. Jen abys počítal s tím, že role fotograf bude mít jinou defaultní stránku než všichni ostatní.

Ano — tohle je teď ideální udělat rovnou čistě, aby se to později snadno rozšířilo pro fotografa.

Doporučení

Neudělal bych jen prosté přepsání `dashboard.php` na `photos.php`, ale zavedl bych jednu centrální funkci, která určí **výchozí stránku po přihlášení podle role**.

Teď bude vracet:

- `photographer` → zatím klidně `/photos.php`
- ostatní → `/photos.php`

Později jen změníš:

- `photographer` → `/photos-status.php`

bez dalších zásahů.

1. Do `inc/functions.php` přidej funkci

Na konec souboru přidej:

```
function default_homepage_for_user(?array $user = null): string
{
```

```
$user = $user ?? current_user();

if (!$user) {
    return '/login.php';
}

$roleCode = (string)($user['role_code'] ?? '');

switch ($roleCode) {
    case 'photographer':
        return '/photos.php'; // později změnit na /photos-status.php
    default:
        . . . . .
}
```

PHP



2. Uprav přesměrování po loginu

V souboru, kde po úspěšném přihlášení děláš něco jako:

PHP



změň na:

PHP



To bude nejspíš v `login.php`.

3. Uprav případný vstupní `index.php`

Jestli máš `index.php`, který přesměrovává přihlášeného uživatele na dashboard, změň:

PHP



na:

⌕ PHP



4. Volitelně uprav menu

Protože jsi říkal, že do budoucna homepage už nebude dashboard, dává smysl časem v `header.php` změnit:

⌕ PHP



na třeba:

⌕ PHP



nebo rovnou:

⌕ PHP



Ale to je kosmetika, může počkat.

Proč je tohle lepší

Ted' si připravíš strukturu pro budoucí stav:

- redaktoři a admini → `photos.php`
- fotografové → později `photos-status.php`

a nebudeš to muset znovu hledat na více místech.

Co hledat

Hledej ve webu řetězec:

⌕ PHP



a oprav hlavně tam, kde je:

- redirect po loginu
- redirect z indexu
- případně redirect po logoutu nechat být

Až budeš chtít, pošli mi login.php a případně index.php a já ti rovnou pošlu přesné finální znění obou souborů.

 Soubor nahrán

index.php vypadá takhle. Prosím pošli mi upravenou verzi

Přemýšlel >

Jasně — tady je upravená verze index.php , aby přihlášené uživatele posílala na výchozí stránku podle role místo natvrdo na dashboard. Vychází z tebou nahraného souboru.

 index

```
<?php
declare(strict_types=1);

require_once __DIR__ . '/config/config.php';
require_once __DIR__ . '/inc/auth.php';
require_once __DIR__ . '/inc/functions.php';

if (is_logged_in()) {
    redirect(default_homepage_for_user(current_user()));
}
```

 PHP



A ještě jedna důležitá věc

Aby to fungovalo, musíš mít v inc/functions.php přidanou funkci:

```
function default_homepage_for_user(?array $user = null): string
{
    $user = $user ?? current_user();

    if (!$user) {
        return '/login.php';
    }
}
```

```

}

$roleCode = (string)($user['role_code'] ?? '');

switch ($roleCode) {
    case 'photographer':
        return '/photos.php'; // později změnit na /photos-status.php
    default:
        return '/photos.php';
}

```

</> PHP



Ted' budou všichni chodit na `photos.php` , ale až bude hotová mobilní read-only stránka pro fotografa, změní se jen ten jeden řádek na `/photos-status.php` .

Zdroje

📁 Soubor nahrán

Zatím to nezafungovalo. Funkci `default_homepage_for_user` v `inc/functions.php` mám. možná ale bude potřeba ještě upravit `login.php` ne?

Přemýšlel >

Ano, přesně tak — `index.php` už je správně, ale `login.php` pořád přesměrovává natvrdo na `/dashboard.php` , a to hned na dvou místech. Je to vidět přímo v nahraném souboru.

📄 login

Takže je potřeba upravit i `login.php` .

Nové celé znění `login.php`

Nahrad' celý soubor tímto:

```

<?php
declare(strict_types=1);

require_once __DIR__ . '/config/config.php';
require_once __DIR__ . '/inc/functions.php';
require_once __DIR__ . '/inc/auth.php';

start_session_if_needed();

```

```
if (is_logged_in()) {  
    redirect(default_homepage_for_user(current_user()));  
}  
  
$error = '';  
  
if (is_guest()) {  
    </> PHP
```

Co bylo špatně

V původním `login.php` byly tyto dva řádky:

```
</> PHP
```

jeden:

- když už je uživatel přihlášený

druhý:

- po úspěšném loginu

A oba bylo potřeba nahradit:

```
</> PHP
```

Co teď očekávat

Po úpravě:

- přímý vstup na `/`
 - `index.php`
 - `photos.php`
- přihlášení přes `/login.php`
 - po loginu také `photos.php`

Až bude jednou hotové `photos-status.php`, fotografovi se změní výchozí stránka jen přes `default_homepage_for_user()` bez dalších zásahů.

Zdroje

Perfektní, funguje. Jdu si commitnout současný stav vývoje.
Jaké další kroky navrhuješ?

Skvělé. Teď je opravdu dobrý moment na commit a krátké nadechnutí — hlavní kostra systému už stojí velmi dobře.

Za mě jsou teď nejlepší další kroky tyto:

1. Sprint: photographer view / `photos-status.php`

Tohle bych posunul úplně nahoru, protože už máš:

- role správně,
- permissions správně,
- workflow správně,
- a chybí už jen specializované UI pro fotografa.

Cíl:

- fotograf vidí jen svoje fotky
- jen read-only
- velmi malé náhledy
- výrazně mobilní layout
- jen to podstatné: náhled + stav

To bude první věc, kterou reálně ocení další testeři.

2. Zachování filtrů a kontextu v galerii

U redaktorů bych pak doladil:

- po návratu z detailu zpět zachovat filtr/page
- po bulk downloadu zachovat aktuální filtr a stránku
- případně tlačítko „moje zamčené“

To je malé, ale UX to zlepší hodně.

3. Dokončení housekeeping kolem download jobů

Funkčně už běží, ale ještě bych dotáhl:

- `download_jobs.status = finished`
- `finished_at`
- případně `failed`
- jednoduchý úklid starých jobů

Není to urgentní pro uživatele, ale je to dobré pro pořádek v DB.

4. Testovací scénáře a pilotní provoz

Teď už má smysl sepsat malý test plan:

- více fotografů nahrává současně
- více redaktorů lockuje současně

- bulk download u více redaktorů
- chování fotografa na mobilu
- mazání / editace uživatelů
- kontrola práv všech rolí

Tohle ti dá velmi rychle jistotu, co ještě drhne.

5. Později: kosmetika a názvosloví

Až po testech bych ladil:

- počestění všech statusů (ready , downloaded)
- menu místo dashboardu třeba „Fotografie“
- flash hlášky po create/edit/delete
- drobné sjednocení textů

Moje doporučení je jasné:

další sprint bych věnoval `photos-status.php` **pro roli photographer.**

To je teď nejlogičtější pokračování, protože:

- funkční jádro už existuje,
- photographer role je definovaná,
- a právě tam ještě chybí specializované UI.

Pošli mi klidně aktuální `inc/photos.php` a já navrhnu první přesnou implementaci `photos-status.php` .

Ano perfektní, pojďme na `photos-status.php`

Přemýšlení po dobu 24s >

Perfektní. Pro roli **photographer** bych to udělal jako opravdu jednoduchou, mobilní, read-only stránku:

- bez filtrů
- bez detailních metadat
- bez akcí
- jen **moje fotky**
- malý náhled
- stav
- u zamčené fotky případně i kdo ji zamknul

Níže máš hotové celé znění `photos-status.php` a k tomu CSS blok do `style.css`.

photos-status.php

Vytvoř nový soubor:

```
<?php
declare(strict_types=1);

require_once __DIR__ . '/config/config.php';
require_once __DIR__ . '/inc/auth.php';
require_once __DIR__ . '/inc/functions.php';
require_once __DIR__ . '/inc/db.php';

require_login();

if (!has_permission('photos.view')) {
    http_response_code(403);
    exit('Přístup odepřen.');
```

`</>` PHP

Co přidat do style.css

Na konec `assets/style.css` přidej:

```
.panel-status {
    max-width: 760px;
    margin: 0 auto;
}

.status-head {
    margin-bottom: 16px;
}

.status-subhead {
    color: var(--text-muted);
    font-size: 14px;
}

.status-photo-list {
    display: flex;
```

Až to budeš chtít aktivovat jako homepage pro fotografa

Pak jen v `default_homepage_for_user()` změníš:

</> PHP



na:

</> PHP



Co tohle přesně splní

Pro fotografa:

- uvidí jen své fotky
- nebude mít žádné akce
- uvidí jednoduchý stav:
 - připraveno
 - zamknuto
 - staženo
 - chyba
- bude to čitelné na mobilu
- u zamčené fotky může vidět i jméno redaktora

A ostatní role se na tuto stránku nedostanou, přesměrují se na běžnou galerii.

Až to nasadíš, můžeme udělat poslední malý krok: přepnout

`default_homepage_for_user()` pro `photographer` na tuhle novou stránku.

Stránka je vytvořená, jen ještě není v menu a když jsem jí zkusil ručně napsat do url skočilo to zpět na `photos.php`

To dává smysl — nejspíš je problém v tom, že `current_user()` u tebe **nemá spolehlivě vyplněné** `role_code`, takže `photos-status.php` vyhodnotí uživatele jako „není photographer“ a pošle ho zpět na `photos.php`.

Uděláme to robustněji:

- `photos-status.php` si roli načte z DB podle aktuálního uživatele
- do menu přidáme položku jen pro fotografa

1. Nové celé znění `photos-status.php`

Nahraď celý soubor tímto:

```
<?php
declare(strict_types=1);

require_once __DIR__ . '/config/config.php';
require_once __DIR__ . '/inc/auth.php';
require_once __DIR__ . '/inc/functions.php';
require_once __DIR__ . '/inc/db.php';

require_login();

if (!has_permission('photos.view')) {
    http_response_code(403);
    exit('Přístup odepřen.');
}

$user = current_user();

</> PHP
```

2. Uprav `header.php`

Přidej do menu položku pro fotografa.

Najdi blok `<nav class="top-nav"> ... </nav>` a nahraď ho tímto:

```
<nav class="top-nav">
    <a href="/dashboard.php">Dashboard</a>
    <a href="/photos.php">Fotografie</a>

    <?php if (($user['role_code'] ?? '') === 'photographer'): ?>
        <a href="/photos-status.php">Moje fotky</a>
    <?php endif; ?>
```

```
<?php if (has_permission('users.manage')): ?>
    <a href="/users.php">Uživatelé</a>
<?php endif; ?>
```

⌄ PHP



3. Když by se menu pořád neukázalo

Pak je to ten samý problém: `header.php` nedostává `role_code` v `$user`.

V tom případě dej vědět a pošlu ti rovnou **robustní verzi** `header.php`, která si roli stejně jako `photos-status.php` načte z DB. To by pak vyřešilo i budoucí věci kolem homepage pro fotografa.

4. Až ověříš, že stránka funguje

Pak změníme v `default_homepage_for_user()`:

⌄ PHP



na:

⌄ PHP



To už bude poslední krok aktivace.

OK, fotografovi se `photos-status` ukáže, ale chtěl bych, aby ho měli v menu dostupní všichni uživatelé. Někdy například uživatel redaktor nebo administrator bude pracovat v terénu jako fotograf a pak bude potřeba, aby si tuhle stránku otevřel (byť nebude jeho defaultní).

Rozumím — to je dobrý požadavek.

Pak tedy `photos-status.php` nebude stránka „jen pro roli `photographer`“, ale obecně stránka „**moje fotky podle mého FTP účtu**“.

To znamená dvě změny:

- v menu ji ukázat všem přihlášeným uživatelům
- v `photos-status.php` zrušit omezení na roli `photographer`

1. Uprav `header.php`

Nahrad' blok `<nav class="top-nav"> ... </nav>` tímto:

```
<nav class="top-nav">
  <a href="/dashboard.php">Dashboard</a>
  <a href="/photos.php">Fotografie</a>
  <a href="/photos-status.php">Moje fotky</a>

  <?php if (has_permission('users.manage')): ?>
    <a href="/users.php">Uživatelé</a>
  <?php endif; ?>
```

⌄ PHP



Tím bude `Moje fotky` vidět všem.

2. Uprav `photos-status.php`

V současné verzi tam máš část, která načte roli z DB a když není `photographer` , přesměruje na `/photos.php` .

Najdi tento blok:

```
$roleCode = (string)($userRow['role_code'] ?? '');
$ftpUser = trim((string)($userRow['ftp_user'] ?? ''));

if ($roleCode !== 'photographer') {
    redirect('/photos.php');
}

if ($ftpUser === '') {
    http_response_code(400);
}
```

⌄ PHP



A nahrad' ho tímto:

```
$ftpUser = trim((string)($userRow['ftp_user'] ?? ''));

if ($ftpUser === '') {

</> PHP
}
```



Tím se zruší omezení jen na fotografa, ale pořád zůstane logika:

- stránka ukazuje jen fotky podle `ftp_user` aktuálního uživatele
- když uživatel nemá FTP účet, dostane smysluplnou chybu

3. Co to bude znamenat prakticky

Pak bude chování takové:

- `photographer`
Moje fotky = jeho hlavní read-only přehled
- `press_operator` , `admin` , `superadmin`
Moje fotky = jejich vlastní přehled fotek podle jejich `ftp_user` , když zrovna fungují i jako fotografové

To přesně odpovídá tomu, co potřebuješ v terénu.

4. Doporučení k homepage

Výchozí homepage bych i tak nechal:

- `photographer` → `photos-status.php`
- ostatní → `photos.php`

To je pořád správně. Jen navíc budou mít všichni v menu i `Moje fotky` .

Když budeš chtít, pošlu ti rovnou celé nové znění `photos-status.php` i `header.php` po této úpravě.

Perfektní, teď je to přesně tak, jak jsem to zamýšlel. Jdu udělat commit změn

Perfektní 👍 teď máš opravdu **uzavřený první plně použitelný milník**:

Hotové:

-  upload fotografií (FTP)

- ☒ watcher + preview
- ☒ galerie redaktorů
- ☒ lock workflow
- ☒ bulk download bez ZIP
- ☒ změna stavů ready → locked → downloaded
- ☒ user management + role
- ☒ read-only mobilní view pro fotografa
- ☒ default homepage podle role
- ☒ zobrazení „zamknuto (kdo)“
- ☒ UX doladění (dimmed locked, progress, refresh...)

To už je reálně nasaditelný PRESS systém.

Co bych dělal dál (doporučené pořadí)

1. Auto-refresh photographer view (velmi užitečné)

Fotograf bude mít otevřený mobil a uvidí:

- ready → zamknuto
- zamknuto → staženo

bez reloadu.

Stačí:

- refresh každých 10–15 s
- jen když je stránka viditelná
- bez skákání scrollu

To je malý kód, velký UX efekt.

2. Zvýraznění změny stavu

Při refreshi:

- ready → zamknuto = žlutý flash
- zamknuto → staženo = zelený flash

Fotograf okamžitě vidí, že redakce stáhla.

3. Počet fotek podle stavu (nahore)

Na `photos-status.php` :

Připraveno: 12

Zamknuto: 3

Staženo: 27



To fotograf ocení okamžitě.

4. "Poslední změna" čas

Do řádku:

staženo – před 12 s

zamknuto – před 1 min



To je super pro live akce.

5. Auto-scroll na nové fotky

Když fotograf nahrává:

- nové fotky nahoře
 - highlight
 - auto-scroll jen při prvním loadu
-

Další větší milestone (po testování)

Sprint: production hardening

- download log (kdo stáhl)
 - audit log
 - cleanup starých preview
 - retry failed preview
 - kontrola orphan RAW bez preview
 - limit bulk download (např. 200)
-

Nejlepší další krok

Já bych teď udělal:

👉 [Auto-refresh photos-status.php](#)

Je to:

- malé
- bezpečné
- okamžitě užitečné
- ideální pro testery

Chceš:

- refresh každých 5 s (rychlé)
- nebo 10–15 s (šetří server)?

Doporučuji **10 s**.

Ano to jsou určitě výborné a užitečné změny. Asi jediná zbytečná je ta flash při změně stavu. Fotograf se bude hlavně věnovat focení, nebude čekat u mobilu, až se něco změní. Jen prosím mysli na minimalizaci datového toku mezi serverem a mobilem. Stejný mobil bude fotograf používat jako hotspot pro posílání fotek na ftp, tak ať mu zbytečně neucpáváme přenosový kanál.

Souhlasím — flash efekt klidně vynecháme. V tomhle provozu je důležitější:

- jednoduchost,
- spolehlivost,
- a hlavně **malý datový provoz**.

A to, co píšeš o hotspotu, je velmi důležité. `photos-status.php` musíme udělat tak, aby:

- se **neobnovovala celá stránka**,
- zbytečně se znovu nestahovaly náhledy,
- a ideálně se přenášely jen malé textové změny stavů.

Co bych tedy navrhl

1. Neobnovovat celou HTML stránku

Místo reloadu celé `photos-status.php` bych udělal malé API, třeba:

```
/api/photos-status.php
```



které vrátí jen lehký JSON:

- id fotky
- status
- locked_by jméno
- uploaded_at

- případně filename

Bez obrázků, bez HTML.

Pak JavaScript na stránce jen:

- porovná stav
- přepíše text u existujících položek
- případně přidá novou položku nahoru

To je datově nejúspornější.

2. Interval spíš 15–20 sekund než 5–10

Kvůli hotspotu bych nešel na 5 s.

Doporučení:

- 15 s jako default
- možná i 20 s

To je pro fotografa úplně dost:

- změny se ukážou rozumně rychle
 - ale síť se nebude zbytečně zatěžovat
-

3. Refresh jen když je stránka aktivní

To je velmi důležité.

Polling dělat jen když:

- stránka je otevřená
- a není schovaná na pozadí

Použít:

JavaScript



Když fotograf přepne jinam, polling se zastaví.

4. Náhledy vůbec znovu nestahovat

To je klíčové.

Jakmile se stránka jednou načte:

- obrázky zůstanou v DOM
- polling už jen mění texty stavů

Tím skoro veškerý provoz po prvním otevření bude jen malý JSON.

5. API vracet jen posledních N fotek

Například:

- posledních 100
- nebo jen ty, které jsou na stránce

A ideálně jen tato pole:

```
[
  {
    "id": 123,
    "status": "locked",
    "locked_by_name": "Jan Novák",
    "uploaded_at": "2026-03-31 14:22:10"
```

JSON



To je velmi malé.

6. Ještě lepší: inkrementální refresh

Ještě úspornější varianta:

stránka si pamatuje:

- poslední updated_at
- nebo poslední id

a API vrátí jen změny od daného okamžiku.

Například:

```
/api/photos-status.php?since=2026-03-31T14:20:00
```



To už je ale druhý krok. Na první verzi bych klidně udělal:

- polling každých 15 s
- celé lehké JSON pole pro posledních třeba 100 fotek

To bude pořád dost úsporné.

Co bych implementoval jako nejbližší krok

Sprint: lightweight auto-refresh pro `photos-status.php`

Obsah:

- nové API `api/photos-status.php`
- polling po 15 s
- jen při `document.visibilityState === 'visible'`
- bez reloadu celé stránky
- bez znovunačítání obrázků
- jen update textu stavu a případně doplnění nových fotek

Co tím získáš

Fotograf:

- otevře stránku
- nechá ji klidně na mobilu
- systém jen občas stáhne malý JSON
- stav se sám mění
- hotspot se zbytečně nezahltí

Co bych neudělal

Ted' bych určitě nedělal:

- WebSockets
- SSE
- agresivní polling
- full page reload
- reload obrázků

To by bylo zbytečné.

Moje doporučení

Pokračoval bych takto:

1. `api/photos-status.php`
2. lehký JS polling v `photos-status.php`
3. interval 15 s
4. refresh jen při viditelné stránce

To je nejlepší poměr přínos / datová úspornost.

Pošli mi aktuální `photos-status.php` a já ti rovnou připravím:

- nové API
- upravený `photos-status.php`
- minimální JS polling bez zbytečného provozu.

Ano to zní velmi rozumně, pojďme to udělat. Interval určitě 20, možná i 30 s

Výborně. Zvolil bych **30 s** jako výchozí interval. Pro tenhle účel je to šetrnější a pořád dost praktické.

Uděláme to takto:

- nový endpoint `api/photos-status.php`
- `photos-status.php` si po prvním načtení drží náhledy v DOM
- každých 30 s si stáhne jen malý JSON
- jen když je stránka viditelná
- bez reloadu celé stránky

1. Nový soubor `api/photos-status.php`

Vytvoř:

```
<?php
declare(strict_types=1);

require_once __DIR__ . '/../config/config.php';
require_once __DIR__ . '/../inc/auth.php';
require_once __DIR__ . '/../inc/functions.php';
require_once __DIR__ . '/../inc/db.php';

require_login();

if (!has_permission('photos.view')) {
    http_response_code(403);
    exit('Přístup odepřen.');
```

</> PHP

2. Nové celé znění `photos-status.php`

Nahrad' celý soubor tímto:

```
<?php
declare(strict_types=1);

require_once __DIR__ . '/config/config.php';
require_once __DIR__ . '/inc/auth.php';
require_once __DIR__ . '/inc/functions.php';
require_once __DIR__ . '/inc/db.php';

require_login();

if (!has_permission('photos.view')) {
    http_response_code(403);
    exit('Přístup odepřen.');
```

PHP

Co to dělá úsporně

Po prvním otevření:

- náhledy se stáhnou jednou
- pak už každých 30 s jde jen malý JSON
- když je stránka na pozadí, polling se zastaví
- žádný full reload

Poznámka

Tahle první verze:

- **mění stavy u již zobrazených fotek**
- **ale nepřidává nové fotky do seznamu**, pokud byly mezitím nově nahrány

To je záměrně kvůli jednoduchosti a malému provozu. Další krok může být doplnění nových položek nahoru, ale stavové změny už teď poběží velmi úsporně.

Až to nasadíš a ověříš, můžeme přidat i velmi lehké doplňování nových fotek bez reloadu.

Ano, přidávání nových fotek do seznamu musíme vyřešit. to je to hlavní co fotografa bude na stage zajímat. Došlo k úspěšnému nahrání mé fotky na ftp?

Ano — přesně tak. U fotografa je nejdůležitější otázka:

„Přistála už moje nová fotka v systému?“

Takže polling musí umět nejen měnit stav už zobrazených položek, ale i **přidávat nové fotky nahoru do seznamu**. A pořád úsporně. Dosavadní `photos-status.php` i API už máme jako základ, takže teď jen rozšíříme JSON a JS.  photo

Nejlepší řešení je:

- každých **30 s**
- jen když je stránka viditelná
- API vrátí malý JSON pro posledních 200 fotek
- JS:
 - aktualizuje stavy existujících položek
 - **doplní nové položky nahoru**
- náhledy se stáhnou jen pro nově přidané fotky

To je přesně ten správný kompromis.

1. Nové celé znění `api/photos-status.php`

Nahrad' celý soubor tímto:

```
<?php
declare(strict_types=1);

require_once __DIR__ . '/../config/config.php';
require_once __DIR__ . '/../inc/auth.php';
require_once __DIR__ . '/../inc/functions.php';
require_once __DIR__ . '/../inc/db.php';

require_login();

if (!has_permission('photos.view')) {
    http_response_code(403);
    exit('Přístup odepřen.');
```

</> PHP

2. Nové celé znění `photos-status.php`

Nahrad' celý soubor tímto:


```
<?php
declare(strict_types=1);

require_once __DIR__ . '/config/config.php';
require_once __DIR__ . '/inc/auth.php';
require_once __DIR__ . '/inc/functions.php';
require_once __DIR__ . '/inc/db.php';

require_login();

if (!has_permission('photos.view')) {
    http_response_code(403);
    exit('Přístup odepřen.');
}

$user = current_user();

</> PHP
```

Co to teď vyřeší

Fotograf uvidí i odpověď na otázku:

- „Objevila se moje nově nahraná fotka?“

protože:

- nová fotka se po dalším pollingu automaticky vloží **nahoru**
- bez reloadu stránky
- bez znovunačítání starých náhledů
- přenesou se jen data pro poslední seznam

Datový provoz

Tahle verze je pořád docela úsporná:

- polling jen po 30 s
- jen při aktivní stránce
- žádný reload HTML
- žádné znovunačítání starých obrázků
- nové obrázky jen pro skutečně nové fotky

Další krok, pokud budeš chtít ještě menší provoz, může být:

- API vracející jen změny od posledního známého ID/času

Ale na první provoz je tohle už rozumně šetrné a funkčně správné.

ještě ti pošlu url loga. Prosím pošli mi návod, jak ho implementovat do headeru (samozřejmě ho na server stáhneme, nebudeme ho tam dávat přímo z toho url. <https://www.clovekavira.cz/themes/cav/logo-cs.svg>

Perfektní — SVG logo je ideální (malé, ostré, rychlé). Uděláme to správně:

- stáhnout na server
- vložit do headeru vedle názvu
- škálovat přes CSS
- klik → homepage

1. Stáhni logo na server

Ulož ho například sem:

❏ Bash



Např.:

❏ Bash



2. Uprav header.php

Najdi část s názvem aplikace, typicky něco jako:

```
<div class="brand">
```

❏ PHP



Nahraď ji tímto:

```
<div class="brand">
  <a href="/" class="brand-link">
    .
  </a>
</div>
```

PHP



Pokud chceš jen logo bez textu, použij:

```
<div class="brand">
  <a href="/" class="brand-link">
    
  </a>
</div>
```

PHP



3. Přidej do style.css

Na konec assets/style.css :

```
.brand-link {
  display: flex;
  align-items: center;
  gap: 10px;
  text-decoration: none;
}

.brand-logo {
  height: 28px;
  width: auto;
  display: block;
}

.brand-text {
  font-weight: 600;
  color: var(--text-main);
}
```

CSS



4. Mobilní varianta (doporučeno)

Aby logo nebylo velké na mobilu:

```
@media (max-width: 640px) {  
  .brand-logo {  
    height: 22px;  
  }  
  
  .brand-text {  
    font-size: 16px;  
  }  
}
```

⌄ CSS



5. Pokud chceš tmavou variantu (SVG umožňuje)

Pokud logo v tmavém headeru nebude dost kontrastní:

```
.brand-logo {  
  height: 28px;  
}
```

⌄ CSS



To z něj udělá bílé logo bez úpravy SVG.

Použij jen pokud je potřeba.

Výsledek

Header pak bude:

[logo Člověk a Víra] Fotografie Moje fotky Uživatelé Odhlásit



A klik na logo → homepage (/ → photos.php).

Pokud mi pošleš aktuální header.php , upravím ho přesně do tvé struktury layoutu.

📁 Soubor nahrán

Zkusil jsem úpravy udělat, ale logo zatím vidět není. Je uložené v assets/logo-cs.svg

Přemýšlel >

Jasně — problém je přímo v `header.php` : odkazuješ na jiný název souboru, než který jsi uložil. V nahraném souboru máš:

❏ PHP



ale píšeš, že soubor je uložený jako:

`assets/logo-cs.svg`



Takže browser hledá `/assets/logo.svg` , který neexistuje.  `header`

Co přesně opravit

V `header.php` změň tento řádek:

❏ PHP



na:

❏ PHP



Správné celé znění `header.php`

```
<?php
declare(strict_types=1);

require_once __DIR__ . '/auth.php';
require_once __DIR__ . '/functions.php';

$user = current_user();
?>
<!doctype html>
<html lang="cs">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title><?= h(APP_NAME) ?></title>
```

```
<link rel="stylesheet" href="/assets/style.css">
```

PHP



Když by logo pořád nebylo vidět

Pak zkontroluj ještě tyto 3 věci:

1. Otevři přímo v browseru

```
https://press.clovekavira.cz/assets/logo-cs.svg
```



Když se zobrazí, cesta je správně.

2. Má webserver právo ten soubor číst

Soubor by měl mít běžně třeba:

```
</> Bash
```



3. CSS logo neskrývá

Zkontroluj, že ve `style.css` máš něco jako:

```
.brand-logo {  
  height: 28px;  
  width: auto;
```

```
</> CSS
```



Ještě jedna drobnost

Až budeš chtít, doporučil bych později změnit položku `Dashboard` v menu na něco jako `Domů` nebo ji úplně zrušit, protože teď už hlavní provoz stejně stojí kolem galerií.

Zdroje

Už je to OK, bylo to tím nesprávným názvem souboru. Teď bych chtěl ještě upravit login stránku. Teď je dialog přihlášení vycentrovaný. Chtěl bych ho umístit na pravou

půlku stránky a na tu levou dál velikou verzi loga a
velikým písmem nápis

PRESS centrum

Člověk a Víra

Aby redaktor snadno našel ve velkém press centru
obrazovku počítače se svým systémem. Asi také bude
dobré tuhle stránku dát bez headeru, aby tam nerušila ta
malá verze loga.

Ano — tohle bude pro provoz velmi dobré. Login uděláme jako **samostatný landing screen bez headeru**:

- vlevo velké logo + velký nápis
- vpravo přihlašovací box
- bez malé hlavičky nahoře

Vycházím z tvého aktuálního `login.php` a `style.css`.  login  style

Nové celé znění `login.php`

Nahraď celý soubor tímto:

```
<?php
declare(strict_types=1);

require_once __DIR__ . '/config/config.php';
require_once __DIR__ . '/inc/functions.php';
require_once __DIR__ . '/inc/auth.php';

start_session_if_needed();

if (is_logged_in()) {
    redirect(default_homepage_for_user(current_user()));
}

$error = '';

if (is_nost()) {
    </> PHP
```



Co přidat do `style.css`

Na konec `assets/style.css` přidej tento blok:

```

.login-page {
  min-height: 100vh;
}

.login-layout {
  min-height: 100vh;
  display: grid;
  grid-template-columns: 1fr 1fr;
}

.login-hero {
  display: flex;
  align-items: center;
  justify-content: center;
  padding: 40px;
}

```

CSS

Co se tím změní

- login.php už **nevolá** inc/header.php **ani** inc/footer.php
- login bude vlastní fullscreen layout
- vlevo:
 - velké logo
 - velký nápis
- vpravo:
 - přihlášení

Ještě drobnost

V tom textu chceš mít odsazení druhého řádku:

```

PRESS centrum
  Člověk a Víra

```

To bych spíš řešil vizuálně jemněji než mezerami. Když ale chceš opravdu druhý řádek trochu posunout, přidej do CSS:

```

.login-hero-title div:last-child {

```

CSS

Na desktopu to vypadá dobře. Na mobilu bych to nechal bez odsazení, takže případně ještě:


```
@media (max-width: 900px) {  
  .login-hero-title div:last-child {  
    margin-left: 0;  
  }  
}
```

CSS



To už je jen kosmetika.

Zdroje

Ano pochopil jsi to správně, chci to mít trochu odsazené. mezery jsem dal jen do promptu, abych to naznačil. Tvé řešení pomocí stylů je samozřejmě to správné.

Perfektní 👍 přesně tak — odsazení přes CSS je správná cesta.

Použijeme **odsazený druhý řádek**, ale jen na desktopu, aby to na mobilu zůstalo čisté.

Přidej do `style.css` :

```
.login-hero-title div:last-child {  
  margin-left: 1.6em;  
}
```

CSS



A pro mobil to zarovnáme zpět:

```
@media (max-width: 900px) {  
  .login-hero-title div:last-child {  
    margin-left: 0;  
  }  
}
```

CSS



Výsledek vizuálně

Desktop:

```
PRESS centrum  
  Člověk a Víra
```



Mobil:



Malé typografické doladění (doporučuji)

Aby to působilo opravdu „brandově“, můžeš ještě přidat:

```
.login-hero-title {  
  font-weight: 700;  
  letter-spacing: -0.02em;  
}  
  
.login-hero-title div:last-child {
```

⌕ CSS



To udělá:

- první řádek dominantní
- druhý řádek jemně doplňující
- vizuálně profesionálnější

Když budeš chtít, můžeme ještě:

- zvětšit logo jen na login page
- přidat jemný gradient pozadí
- nebo udělat „event mode“ (velké pro stage)

Ukázalo se, že v user managementu je jedna zásadní chyba. Když vytvoříme/změníme usera, je nutné mu také vytvořit/zkontrolovat adresář v kořeni ftp a nastavit práva chown www-data:www-data

Ano — tohle je opravdu zásadní a je dobře, že se to ukázalo teď. Samotný záznam v DB nestačí. Pro funkčního FTP uživatele musí při create/edit proběhnout i **synchronizace filesystemu**:

- adresář musí existovat
- musí být ve správné cestě
- musí mít správného ownera `www-data:www-data`

Je tu ale jeden důležitý technický háček:

Důležitá realita

PHP běžící pod Apache jako `www-data` **většinou nemůže zavolat** `chown()` **na** `www-data:www-data` , pokud soubor/adresář nevlastní nebo pokud proces nemá zvýšená práva. A už vůbec ne bezpečně u všech scénářů.

Takže správné řešení je:

- z PHP zavolat **malý root-owned helper skript**
- ten provede:
 - `mkdir -p`
 - `chown www-data:www-data`
 - případně `chmod`

To je mnohem spolehlivější než snažit se vše dělat přímo v PHP.

Co navrhuju

1. Přidat helper skript na server

Například:

```
/usr/local/bin/press-ensure-ftp-dir.sh
```



Obsah:

```
#!/bin/bash
set -euo pipefail

DIR="${1:-}"

if [[ -z "$DIR" ]]; then
    echo "Missing directory path" >&2
    exit 1
fi

case "$DIR" in
    /var/www/press/ftp/*) ;;
    *)
        echo "Directory outside allowed base path" >&2
        exit 1
    ;;
)
```

⌕ Bash



Pak:

 Bash



2. Povolit spuštění přes sudo jen pro tenhle skript

Otevři:

ChatGPT 

a přidej:

```
www-data ALL=(root) NOPASSWD: /usr/local/bin/press-ensure-ftp-dir.sh
```



Tím dovolíš Apache/PHP spustit jen ten jeden konkrétní skript jako root.

3. Přidat PHP funkci do inc/users.php

Na konec inc/users.php přidej:

```
function users_ensure_ftp_directory(string $homedir): void
{
    $homedir = trim($homedir);

    if ($homedir === '') {
        throw new RuntimeException('Chybí home directory.');
```

 PHP



4. Zavolat ji po create

Ve funkci `users_create()` po `execute()` a před `return` přidej:

⌌ PHP



Takže konec funkce bude:

```
$stmt->execute([
    ':jmeno'          => $data['jmeno'],
    ':prijmeni'       => $data['prijmeni'],
    ':user'           => $data['user'],
    ':pass_hash'       => password_hash($data['password'], PASSWORD_BCRYPT),
    ':ftp_user'        => $data['ftp_user'],
    ':ftp_pass_hash'   => password_hash($data['ftp_password'], PASSWORD_BCRYPT),
    ':homedir'         => $data['homedir'],
    ':role_id'         => $data['role_id'],
]);

users_ensure_ftp_directory($data['homedir']);
```

⌌ PHP



5. Zavolat ji po editaci

Do `users_update()` po `$stmt->execute($params);` přidej:

⌌ PHP



Takže konec:

```
$stmt = $pdo->prepare($sql);
$stmt->execute($params);
```

⌌ PHP



Ještě lepší bezpečnost

Protože home dir má být u vás vždy:

```
/var/www/press/ftp/<ftp_user>
```



doporučil bych časem vůbec nedovolit libovolné zadávání `homedir` , ale odvozovat ho automaticky z `ftp_user` .

Tedy:

- ve formuláři ho klidně zobrazit
- ale při uložení ho vynutit jako:

```
</> PHP
```



Tím se vyhneš:

- chybám v ručním zadání
- bezpečnostním problémům
- nesouladu mezi FTP loginem a cestou

To bych doporučil jako další malý cleanup.

Co když nechceš sudo helper

Pak máš jen méně spolehlivé varianty:

- spoléhat, že adresáře budou vznikat už pod `www-data`
- a že `chown()` nebude potřeba

Ale protože výslovně chceš `chown www-data:www-data` , je helper skript správná cesta.

Praktický minimální plán

Udělal bych to teď takto:

1. vytvořit `/usr/local/bin/press-ensure-ftp-dir.sh`
2. přidat pravidlo do `sudoers`
3. doplnit `users_ensure_ftp_directory()` do `inc/users.php`
4. volat ji po create i update

Tím se chyba opraví systémově.

Pošli mi aktuální `inc/users.php` , když chceš, a já ti ho vrátím už celý upravený včetně této synchronizace.